WhoPlay — Guia completo de construção em Flutter

Oct 27, 2026 · @Sebastião Cardoso

Visão geral

Seguindo este guia de cima para baixo, recriam a app WhoPlay em Flutter, igual ao mockup *WhoPlay Mockups 2.html*: 10 ecrãs, 22 ficheiros Dart, cerca de 2 000 linhas. Cada ficheiro aparece completo, pronto a copiar, com uma explicação antes do código.

O WhoPlay liga pessoas para praticar desporto com base na localização, no nível, na disponibilidade e nos objetivos. O slogan é *Encontra. Marca. Joga.*

Os 10 ecrãs

Nº	Ecrã	Ficheiro	Tem barra de navegação?
01	Login	`login_screen.dart`	Não
02	Onboarding — desportos	`onboarding_screen.dart`	Não
03	Perfil	`profile_screen.dart`	Sim
04	Encontrar jogadores	`find_players_screen.dart`	Sim
05	Perfil do jogador	`player_profile_screen.dart`	Não
06	Convites e avisos	`invites_screen.dart`	Sim
07	Criar jogo	`create_game_screen.dart`	Não
08	Mapa	`map_screen.dart`	Sim
09	Detalhe do jogo	`game_detail_screen.dart`	Não
10	Check-in por QR Code	`checkin_screen.dart`	Não

Estrutura final de pastas

No fim do guia, a pasta `lib/` fica assim:

```
whoplay/
├── pubspec.yaml
├── analysis_options.yaml
├── lib/
│   ├── main.dart                    ← arranque da app
│   ├── theme/
│   │   ├── app_colors.dart          ← paleta de cores
│   │   └── app_theme.dart           ← fontes e tema
│   ├── widgets/                     ← peças reutilizáveis
│   │   ├── app_icons.dart
│   │   ├── logo.dart
│   │   ├── buttons.dart
│   │   ├── fields.dart
│   │   ├── common.dart
│   │   └── bottom_nav.dart
│   ├── data/
│   │   └── mock_data.dart           ← dados de exemplo
│   └── screens/                     ← um ficheiro por ecrã
│       ├── home_shell.dart
│       ├── login_screen.dart
│       ├── onboarding_screen.dart
│       ├── profile_screen.dart
│       ├── find_players_screen.dart
│       ├── player_profile_screen.dart
│       ├── invites_screen.dart
│       ├── create_game_screen.dart
│       ├── map_screen.dart
│       ├── game_detail_screen.dart
│       └── checkin_screen.dart
└── test/
    └── widget_test.dart
```

Pré-requisitos

- **Flutter SDK 3.22 ou mais recente** (foi testado com o Flutter 3.47 e o Dart 3.13). Para confirmar, corram `flutter --version`.
- **O `flutter` no PATH.** No Windows, juntem `C:\caminho\para\flutter\bin` à variável Path. No Git Bash, `echo 'export PATH="/c/src/flutter/bin:$PATH"' >> ~/.bashrc`.
- **Editor:** VS Code com as extensões *Flutter* e *Dart*, ou o Android Studio.
- **Um dispositivo para testar:** o Chrome chega para começar. Para Android precisam do Android Studio com um emulador; para iOS, de um Mac com o Xcode.

- **Internet na primeira execução:** as fontes Google são descarregadas quando a app abre pela primeira vez.

Como usar este guia

1. Façam os passos 1 a 5 pela ordem: projeto, tema, componentes, dados e navegação.
2. Depois criem os ecrãs 01 a 10. Cada um depende apenas dos passos anteriores.
3. Os ecrãs importam-se uns aos outros, por isso a app só compila quando os 22 ficheiros existirem todos. Se quiserem testar a meio, comentem os `import` e as rotas dos ecrãs que ainda não fizeram.
4. No fim, corram `flutter analyze`; deve responder *No issues found!*

Passo 1 — Criar o projeto

Criem o projeto com o `flutter create`, substituam o `pubspec.yaml` e instalem os 3 pacotes externos.

1.1 Gerar o projeto

Na pasta onde querem o projeto, corram:

```
flutter create whoplay --org com.whoplay --platforms android,ios,web
cd whoplay
```

O comando cria as pastas `android/`, `ios/`, `web/`, `lib/` e `test/`. Apaguem o conteúdo de `lib/main.dart` (é reescrito no passo 5) e o ficheiro `test/widget_test.dart` (é reescrito na secção final).

1.2 `pubspec.yaml`

Substituam o ficheiro inteiro por este. Os pacotes que vêm de fora são:

Pacote	Para que serve
`google_fonts`	Fontes *Bricolage Grotesque* (títulos) e *DM Sans* (texto), as mesmas do mockup
`flutter_svg`	Desenhar os ícones, o logo e o mapa a partir do SVG original
`qr_flutter`	Gerar o QR Code do check-in
`flutter_localizations`	Calendário e relógio em português (já vem com o Flutter)

```
name: whoplay
description: "WhoPlay — Encontra. Marca. Joga."
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: '>=3.3.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  flutter_localizations:
    sdk: flutter
  google_fonts: ^6.2.1
  flutter_svg: ^2.0.10
  qr_flutter: ^4.1.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^4.0.0

flutter:
  uses-material-design: true
```

1.3 `analysis_options.yaml`

Ativam as regras de estilo oficiais do Flutter e mandam o analisador ignorar as pastas das plataformas.

```
analyzer:
  exclude:
    - build/**
    - android/**
    - ios/**
    - web/**
include: package:flutter_lints/flutter.yaml
```

1.4 Instalar os pacotes e criar as pastas

```
flutter pub get
mkdir -p lib/theme lib/widgets lib/data lib/screens
```

No PowerShell, a segunda linha é `mkdir lib/theme, lib/widgets, lib/data, lib/screens`.

Passo 2 — Tema: cores e tipografia

Todas as cores e fontes da app ficam em 2 ficheiros. Nenhum ecrã escreve um código de cor diretamente: usa sempre `AppColors.x` e `AppText.y`, e assim mudar uma cor muda-a na app toda.

2.1 A paleta do mockup

Nome no código	Hex	Onde se usa
`background`	#F3F5F0	Fundo de todos os ecrãs
`ink`	#10201A	Texto principal, contornos dos botões, pílula selecionada
`muted`	#55635C	Texto secundário, ícones inativos
`primary`	#1F7A4D	Botões principais, links, item ativo da navegação
`forest`	#123D29	Cabeçalho do login, cartão do perfil, avatares
`forestLight`	#2C5A43	Calha das barras de compatibilidade, botão voltar no detalhe do jogo
`night`	#0E1A14	Fundo do ecrã de check-in
`nightLight`	#22362C	Botão fechar do check-in
`lime`	#C9F25C	Cor de destaque: seleções, percentagens, logo

Nome no código	Hex	Onde se usa
`border`	#DDE3DA	Contorno dos cartões e separadores
`inputBorder`	#B9C4BA	Contorno dos campos e das opções não selecionadas
`track`	#CDD6CB	Passos por fazer no onboarding, avisos lidos
`chip`	#E6ECE3	Etiquetas cinzentas ("Iniciante", "A aguardar")
`softText`	#C9D3CB	Texto claro sobre fundo escuro
`mapLand`	#E2E9DD	Terreno do mapa
`white`	#FFFFFF	Cartões e campos

2.2 `lib/theme/app_colors.dart`

Em Flutter uma cor escreve-se `Color(0xFFRRGGBB)`: o `FF` inicial é a opacidade (100%) e o resto é o hex do mockup. `abstract final class` impede que alguém crie objetos desta classe, que serve só para agrupar as constantes.

```
import 'package:flutter/material.dart';

/// Paleta tirada diretamente do mockup (WhoPlay Mockups 2.html).
abstract final class AppColors {
  static const background = Color(0xFFF3F5F0);
  static const ink = Color(0xFF10201A);
  static const muted = Color(0xFF55635C);
  static const primary = Color(0xFF1F7A4D);
  static const forest = Color(0xFF123D29);
  static const forestLight = Color(0xFF2C5A43);
  static const night = Color(0xFF0E1A14);
  static const nightLight = Color(0xFF22362C);
  static const lime = Color(0xFFC9F25C);
  static const border = Color(0xFFDDE3DA);
  static const inputBorder = Color(0xFFB9C4BA);
  static const track = Color(0xFFCDD6CB);
  static const chip = Color(0xFFE6ECE3);
  static const softText = Color(0xFFC9D3CB);
  static const mapLand = Color(0xFFE2E9DD);
  static const white = Color(0xFFFFFFFF);
}
```

2.3 `lib/theme/app_theme.dart`

Este ficheiro tem 2 partes:

- **`AppText`** cria estilos de texto. `AppText.display(28)` dá a fonte de títulos (Bricolage Grotesque) e `AppText.body(16)` a de texto corrido (DM Sans). Os parâmetros correspondem ao CSS do mockup: `font-size` → `size`, `font-weight: 700` → `FontWeight.w700`, `letter-spacing: -0.5px` → `letterSpacing: -0.5`, `line-height: 1.4` → `height: 1.4`.
- **`AppTheme`** monta o `ThemeData` (fundo, cor principal, DM Sans como fonte por omissão e estilo das mensagens *SnackBar*). Define também os 2 estilos da barra de estado: ícones escuros para ecrãs claros e ícones claros para ecrãs escuros.

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:google_fonts/google_fonts.dart';

import 'app_colors.dart';

/// Tipografia: Bricolage Grotesque para títulos, DM Sans para o resto.
abstract final class AppText {
  static TextStyle display(
    double size, {
    FontWeight weight = FontWeight.w700,
    Color color = AppColors.ink,
    double? letterSpacing,
    double? height,
  }) =>
      GoogleFonts.getFont(
        'Bricolage Grotesque',
        fontSize: size,
        fontWeight: weight,
        color: color,
        letterSpacing: letterSpacing,
        height: height,
      );

  static TextStyle body(
    double size, {
    FontWeight weight = FontWeight.w400,
    Color color = AppColors.ink,
    double? letterSpacing,
    double? height,
  }) =>
      GoogleFonts.dmSans(
        fontSize: size,
```

```
        fontWeight: weight,
        color: color,
        letterSpacing: letterSpacing,
        height: height,
      );
}

abstract final class AppTheme {
  static ThemeData get light {
    final base = ThemeData(
      useMaterial3: true,
      scaffoldBackgroundColor: AppColors.background,
      colorScheme: ColorScheme.fromSeed(
        seedColor: AppColors.primary,
        primary: AppColors.primary,
        surface: AppColors.background,
      ),
    );
    return base.copyWith(
      textTheme: GoogleFonts.dmSansTextTheme(base.textTheme).apply(
        bodyColor: AppColors.ink,
        displayColor: AppColors.ink,
      ),
      snackBarTheme: SnackBarThemeData(
        backgroundColor: AppColors.ink,
        contentTextStyle: AppText.body(15, weight: FontWeight.w600, color:
AppColors.white),
        behavior: SnackBarBehavior.floating,
        shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(12)),
      ),
    );
  }

  static const darkStatusIcons = SystemUiOverlayStyle(
    statusBarColor: Colors.transparent,
    statusBarIconBrightness: Brightness.dark,
    statusBarBrightness: Brightness.light,
  );

  static const lightStatusIcons = SystemUiOverlayStyle(
    statusBarColor: Colors.transparent,
    statusBarIconBrightness: Brightness.light,
    statusBarBrightness: Brightness.dark,
  );
}
```

Passo 3 — Componentes partilhados

As peças que se repetem nos ecrãs (botões, campos, cartões, pílulas, avatares, barra de navegação) são feitas uma vez em `lib/widgets/`. Os ecrãs limitam-se a montá-las.

Ficheiro	O que tem
`app_icons.dart`	Os 10 ícones do mockup e o widget `SvgIcon`
`logo.dart`	`LogoMark` (símbolo) e `WhoPlayLogo` (símbolo + nome)
`buttons.dart`	`AppButton` (normal, `.small`, `.pill`) e `CircleIconButton`
`fields.dart`	`FieldLabel`, `FieldBox`, `LabeledField`, `AppTextField`, `AppDropdown`
`common.dart`	Ajudas de layout e os blocos visuais: `Avatar`, `Tag`, `SurfaceCard`, `ListCard`, `PillChip`, `ChoiceTile`, `ThinBar`…
`bottom_nav.dart`	`AppBottomNav`, a barra de baixo com 5 entradas

3.1 `lib/widgets/app_icons.dart`

Os ícones são os SVG copiados do mockup: cada um é só o "miolo" do SVG (paths e círculos), num `enum` para não haver gralhas nos nomes. O `SvgIcon` junta o cabeçalho `<svg>` com a espessura do traço pedida e pinta-o na cor dada (através de `currentColor` e do `SvgTheme`). Uso: `SvgIcon(AppIcon.pin, size: 16, color: AppColors.white)`.

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

/// Ícones do mockup (SVG 24x24, traço arredondado).
enum AppIcon { users, pin, plus, bell, user, check, clock, back, star, close
}

const _paths = <AppIcon, String>{
  AppIcon.users:
      '<path d="M16 19v-1a4 4 0 0 0-4-4H7a4 4 0 0 0-4 4v1"/><circle cx="9.5"
cy="7" r="3.5"/><path d="M21 19v-1a4 4 0 0 0-3-3.8M15.5 3.7a3.5 3.5 0 0 1 0
6.6"/>',
  AppIcon.pin:
      '<path d="M12 21s7-6.2 7-11a7 7 0 1 0-14 0c0 4.8 7 11 7 11z"/><circle
cx="12" cy="10" r="2.5"/>',
```

```
  AppIcon.plus: '<path d="M12 5v14M5 12h14"/>',
  AppIcon.bell:
      '<path d="M6 9a6 6 0 1 1 12 0c0 6 2.5 7 2.5 7h-17S6 15 6 9z"/><path
d="M10 20a2 2 0 0 0 4 0"/>',
  AppIcon.user:
      '<circle cx="12" cy="8" r="4"/><path d="M4 21v-1a6 6 0 0 1 6-6h4a6 6 0
0 1 6 6v1"/>',
  AppIcon.check: '<path d="M5 12.5l4.5 4.5L19 7.5"/>',
  AppIcon.clock: '<circle cx="12" cy="12" r="9"/><path d="M12 7v5l3 2"/>',
  AppIcon.back: '<path d="M15 5l-7 7 7 7"/>',
  AppIcon.star:
      '<path fill="currentColor" d="M12 3.5l2.6 5.4 5.9.8-4.3 4.1 1 5.8L12
16.9 6.8 19.6l1-5.8L3.5 9.7l5.9-.8z"/>',
  AppIcon.close: '<path d="M6 6l12 12M18 6L6 18"/>',
};

class SvgIcon extends StatelessWidget {
  const SvgIcon(
    this.icon, {
    super.key,
    this.size = 22,
    this.color = const Color(0xFF10201A),
    this.strokeWidth = 2,
  });

  final AppIcon icon;
  final double size;
  final Color color;
  final double strokeWidth;

  @override
  Widget build(BuildContext context) {
    final svg = '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" '
        'stroke-width="$strokeWidth" stroke-linecap="round" stroke-
linejoin="round" '
        'xmlns="http://www.w3.org/2000/svg">${_paths[icon]}</svg>';
    return SvgPicture.string(
      svg,
      width: size,
      height: size,
      theme: SvgTheme(currentColor: color),
    );
  }
}
```

3.2 `lib/widgets/logo.dart`

O símbolo do WhoPlay é uma bola (a cabeça) sobre um "W" (os braços no ar). Por omissão, `WhoPlayLogo` desenha a versão do login: símbolo lima com 60 px e o nome branco com 52 px.

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';

const _logoSvg = '<svg viewBox="0 0 24 24" fill="none"
xmlns="http://www.w3.org/2000/svg">'
    '<circle cx="12" cy="4.2" r="2.4" fill="currentColor"/>'
    '<path d="M4 9.5l3.6 10.5L12 11.5l4.4 8.5L20 9.5" stroke="currentColor" '
    'stroke-width="3.2" stroke-linecap="round" stroke-linejoin="round"/>
</svg>';

/// Símbolo do WhoPlay (jogador a celebrar / "W").
class LogoMark extends StatelessWidget {
  const LogoMark({super.key, this.size = 60, this.color = AppColors.lime});

  final double size;
  final Color color;

  @override
  Widget build(BuildContext context) => SvgPicture.string(
        _logoSvg,
        width: size,
        height: size,
        theme: SvgTheme(currentColor: color),
      );
}

/// Símbolo + palavra "WhoPlay".
class WhoPlayLogo extends StatelessWidget {
  const WhoPlayLogo({
    super.key,
    this.markSize = 60,
    this.fontSize = 52,
    this.markColor = AppColors.lime,
    this.textColor = AppColors.white,
  });

  final double markSize;
```

```
  final double fontSize;
  final Color markColor;
  final Color textColor;

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        LogoMark(size: markSize, color: markColor),
        const SizedBox(width: 10),
        Text(
          'WhoPlay',
          style: AppText.display(
            fontSize,
            weight: FontWeight.w800,
            color: textColor,
            letterSpacing: -1.5,
            height: 1,
          ),
        ),
      ],
    );
  }
}
```

3.3 `lib/widgets/buttons.dart`

O `AppButton` tem 3 tamanhos, um para cada tipo de botão do mockup:

Construtor	Altura	Raio	Letra	Onde aparece
`AppButton(...)`	56	14	17	"Entrar", "Continuar", "Criar jogo"…
`AppButton.small(...)`	44	12	15	"Ver perfil", "Convidar", "Aceitar"…
`AppButton.pill(...)`	44	22	15	"Editar" no perfil

`outlined: true` troca o fundo verde por um contorno de 1,5 px. Com `onPressed: null` o botão deixa de responder ao toque. O `CircleIconButton` é o círculo de 44 px usado para voltar e fechar.

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
```

```
/// Botão principal (verde) ou contornado, como no mockup.
class AppButton extends StatelessWidget {
  const AppButton({
    super.key,
    required this.label,
    required this.onPressed,
    this.outlined = false,
    this.height = 56,
    this.radius = 14,
    this.fontSize = 17,
    this.foreground,
    this.expand = true,
    this.horizontalPadding = 0,
  });

  /// Botão pequeno usado nos cartões (44px, raio 12, 15px).
  const AppButton.small({
    super.key,
    required this.label,
    required this.onPressed,
    this.outlined = false,
    this.foreground,
  })  : height = 44,
        radius = 12,
        fontSize = 15,
        expand = true,
        horizontalPadding = 0;

  /// Botão em forma de pílula (ex.: "Editar").
  const AppButton.pill({
    super.key,
    required this.label,
    required this.onPressed,
    this.foreground,
  })  : outlined = true,
        height = 44,
        radius = 22,
        fontSize = 15,
        expand = false,
        horizontalPadding = 18;

  final String label;
  final VoidCallback? onPressed;
  final bool outlined;
  final double height;
```

```
  final double radius;
  final double fontSize;
  final Color? foreground;
  final bool expand;
  final double horizontalPadding;

  @override
  Widget build(BuildContext context) {
    final fg = foreground ?? (outlined ? AppColors.ink : AppColors.white);
    return SizedBox(
      height: height,
      width: expand ? double.infinity : null,
      child: Material(
        color: outlined ? Colors.transparent : AppColors.primary,
        clipBehavior: Clip.antiAlias,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(radius),
          side: outlined ? BorderSide(color: fg, width: 1.5) :
BorderSide.none,
        ),
        child: InkWell(
          onTap: onPressed,
          child: Padding(
            padding: EdgeInsets.symmetric(horizontal: horizontalPadding),
            child: Center(
              widthFactor: expand ? null : 1,
              child: Text(
                label,
                style: AppText.body(fontSize, weight: FontWeight.w700, color:
fg),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

/// Botão redondo de 44px (voltar / fechar).
class CircleIconButton extends StatelessWidget {
  const CircleIconButton({
    super.key,
    required this.child,
    required this.onPressed,
    this.background = AppColors.white,
```

```
    this.bordered = true,
    this.semanticLabel,
  });

  final Widget child;
  final VoidCallback? onPressed;
  final Color background;
  final bool bordered;
  final String? semanticLabel;

  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      label: semanticLabel,
      child: Material(
        color: background,
        shape: CircleBorder(
          side: bordered ? const BorderSide(color: AppColors.border) :
BorderSide.none,
        ),
        clipBehavior: Clip.antiAlias,
        child: InkWell(
          onTap: onPressed,
          child: SizedBox(width: 44, height: 44, child: Center(child:
child)),
        ),
      ),
    );
  }
}
```

3.4 `lib/widgets/fields.dart`

Todos os campos do mockup são uma caixa branca com 52 px de altura, contorno `inputBorder` e raio 12. Essa caixa é o `FieldBox`. O `LabeledField` põe um rótulo de 14 px por cima, a 6 px. O `AppTextField` aceita um `initialValue` (texto fixo) ou um `controller` (texto que o código muda, como as datas do ecrã Criar jogo). O `AppDropdown` é o seletor de desporto.

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
```

```
class FieldLabel extends StatelessWidget {
  const FieldLabel(this.text, {super.key});

  final String text;

  @override
  Widget build(BuildContext context) =>
      Text(text, style: AppText.body(14, weight: FontWeight.w600));
}

/// Caixa branca de 52px com contorno, base de todos os campos.
class FieldBox extends StatelessWidget {
  const FieldBox({super.key, required this.child, this.horizontalPadding =
16});

  final Widget child;
  final double horizontalPadding;

  @override
  Widget build(BuildContext context) {
    return Container(
      height: 52,
      alignment: Alignment.centerLeft,
      padding: EdgeInsets.symmetric(horizontal: horizontalPadding),
      decoration: BoxDecoration(
        color: AppColors.white,
        border: Border.all(color: AppColors.inputBorder),
        borderRadius: BorderRadius.circular(12),
      ),
      child: child,
    );
  }
}

class LabeledField extends StatelessWidget {
  const LabeledField({super.key, required this.label, required this.child});

  final String label;
  final Widget child;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      mainAxisSize: MainAxisSize.min,
      children: [FieldLabel(label), const SizedBox(height: 6), child],
```

```
    );
  }
}

class AppTextField extends StatelessWidget {
  const AppTextField({
    super.key,
    required this.label,
    this.initialValue,
    this.controller,
    this.obscureText = false,
    this.keyboardType,
    this.weight = FontWeight.w500,
    this.readOnly = false,
    this.onTap,
  });

  final String label;
  final String? initialValue;
  final TextEditingController? controller;
  final bool obscureText;
  final TextInputType? keyboardType;
  final FontWeight weight;
  final bool readOnly;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    return LabeledField(
      label: label,
      child: FieldBox(
        child: TextFormField(
          initialValue: controller == null ? initialValue : null,
          controller: controller,
          obscureText: obscureText,
          keyboardType: keyboardType,
          readOnly: readOnly,
          onTap: onTap,
          cursorColor: AppColors.primary,
          style: AppText.body(16, weight: weight),
          decoration: const InputDecoration.collapsed(hintText: null),
        ),
      ),
    );
  }
}
```

```
class AppDropdown<T> extends StatelessWidget {
  const AppDropdown({
    super.key,
    required this.label,
    required this.value,
    required this.items,
    required this.onChanged,
  });

  final String label;
  final T value;
  final Map<T, String> items;
  final ValueChanged<T?> onChanged;

  @override
  Widget build(BuildContext context) {
    return LabeledField(
      label: label,
      child: FieldBox(
        horizontalPadding: 12,
        child: DropdownButtonHideUnderline(
          child: DropdownButton<T>(
            value: value,
            isExpanded: true,
            borderRadius: BorderRadius.circular(12),
            dropdownColor: AppColors.white,
            icon: const Icon(Icons.keyboard_arrow_down_rounded, color:
AppColors.ink),
            style: AppText.body(16, weight: FontWeight.w600),
            items: [
              for (final e in items.entries)
                DropdownMenuItem(value: e.key, child: Text(e.value)),
            ],
            onChanged: onChanged,
          ),
        ),
      ),
    );
  }
}
```

3.5 `lib/widgets/common.dart`

É o ficheiro mais usado. As 2 ajudas de layout mais importantes são estas:

- **`topInset(context, valor)`** converte o espaço de cima do mockup para o telemóvel real. O mockup tem 844 px de altura e conta com cerca de 44 px de barra de estado dentro do `padding-top`, por isso um `padding-top: 52px` passa a ser "barra de estado real + 8".
- **`FillScrollView`** é o equivalente do `margin-top: auto` do CSS. O conteúdo ocupa o ecrã todo, um `Spacer()` empurra o botão principal para o fundo e, em telemóveis pequenos, a página passa a fazer scroll em vez de cortar o conteúdo.

Os restantes são blocos visuais:

Widget	Mockup
`ScreenTitle`	Título de ecrã, 28 px, Bricolage
`SectionLabel` / `Section`	Títulos em maiúsculas ("DESPORTOS E NÍVEL") com o conteúdo 10 px abaixo
`Avatar`	Círculo com a inicial (verde-floresta e lima por omissão)
`Tag`	Etiqueta arredondada ("Intermédio", "91%", "A aguardar")
`SurfaceCard`	Cartão branco com contorno
`ListCard`	Cartão com linhas separadas por um traço de 1 px
`PillChip`	Filtro em pílula, escuro quando está selecionado
`ChoiceTile`	Opção de nível ou formato, lima com contorno de 2 px quando está selecionada
`ThinBar`	Barra de progresso de 6 px

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';

/// O mockup tem 390x844 com ~44px de barra de estado incluídos no padding
/// superior. Aqui convertemos esse valor para o inset real do dispositivo.
double topInset(BuildContext context, double mockupTop) =>
    MediaQuery.paddingOf(context).top + mockupTop - 44;

double bottomInset(BuildContext context) =>
MediaQuery.paddingOf(context).bottom;
```

```
void showInfo(BuildContext context, String message) {
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar()
    ..showSnackBar(SnackBar(content: Text(message)));
}

/// Conteúdo que preenche o ecrã (para botões "colados" ao fundo) mas que
/// faz scroll em ecrãs mais pequenos.
class FillScrollView extends StatelessWidget {
  const FillScrollView({super.key, required this.child, this.padding =
EdgeInsets.zero});

  final Widget child;
  final EdgeInsetsGeometry padding;

  @override
  Widget build(BuildContext context) {
    return CustomScrollView(
      slivers: [
        SliverFillRemaining(
          hasScrollBody: false,
          child: Padding(padding: padding, child: child),
        ),
      ],
    );
  }
}

class ScreenTitle extends StatelessWidget {
  const ScreenTitle(this.text, {super.key});

  final String text;

  @override
  Widget build(BuildContext context) =>
      Text(text, style: AppText.display(28, letterSpacing: -0.5));
}

class SectionLabel extends StatelessWidget {
  const SectionLabel(this.text, {super.key, this.color = AppColors.muted,
this.letterSpacing = 0.8});

  final String text;
  final Color color;
  final double letterSpacing;
```

```
  @override
  Widget build(BuildContext context) => Text(
        text.toUpperCase(),
        style: AppText.body(13, weight: FontWeight.w700, color: color,
letterSpacing: letterSpacing),
      );
}

/// Secção com título em maiúsculas e conteúdo a 10px.
class Section extends StatelessWidget {
  const Section({super.key, required this.title, required this.child});

  final String title;
  final Widget child;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [SectionLabel(title), const SizedBox(height: 10), child],
    );
  }
}

class Avatar extends StatelessWidget {
  const Avatar(
    this.letter, {
    super.key,
    this.size = 48,
    this.fontSize = 20,
    this.background = AppColors.forest,
    this.foreground = AppColors.lime,
    this.borderWidth = 0,
  });

  final String letter;
  final double size;
  final double fontSize;
  final Color background;
  final Color foreground;
  final double borderWidth;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: size,
```

```
      height: size,
      alignment: Alignment.center,
      decoration: BoxDecoration(
        color: background,
        shape: BoxShape.circle,
        border: borderWidth > 0 ? Border.all(color: AppColors.white, width:
borderWidth) : null,
      ),
      child: Text(
        letter,
        style: AppText.display(fontSize, weight: FontWeight.w800, color:
foreground, height: 1),
      ),
    );
  }
}

/// Etiqueta arredondada (nível, compatibilidade, estado...).
class Tag extends StatelessWidget {
  const Tag(
    this.text, {
    super.key,
    this.background = AppColors.chip,
    this.foreground = AppColors.ink,
    this.fontSize = 14,
    this.padding = const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
    this.radius = 12,
    this.display = false,
  });

  final String text;
  final Color background;
  final Color foreground;
  final double fontSize;
  final EdgeInsets padding;
  final double radius;
  final bool display;

  @override
  Widget build(BuildContext context) {
    final style = display
        ? AppText.display(fontSize, weight: FontWeight.w800, color:
foreground)
        : AppText.body(fontSize, weight: FontWeight.w700, color: foreground);
    return Container(
      padding: padding,
```

```
      decoration: BoxDecoration(color: background, borderRadius:
BorderRadius.circular(radius)),
      child: Text(text, style: style, softWrap: false),
    );
  }
}

/// Cartão branco com contorno.
class SurfaceCard extends StatelessWidget {
  const SurfaceCard({super.key, required this.child, this.padding =
EdgeInsets.zero, this.radius = 16});

  final Widget child;
  final EdgeInsetsGeometry padding;
  final double radius;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: padding,
      decoration: BoxDecoration(
        color: AppColors.white,
        border: Border.all(color: AppColors.border),
        borderRadius: BorderRadius.circular(radius),
      ),
      child: child,
    );
  }
}

/// Cartão branco com linhas separadas por um traço de 1px.
class ListCard extends StatelessWidget {
  const ListCard({super.key, required this.children, this.radius = 16});

  final List<Widget> children;
  final double radius;

  @override
  Widget build(BuildContext context) {
    return SurfaceCard(
      radius: radius,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          for (var i = 0; i < children.length; i++) ...[
            if (i > 0) const ColoredBox(color: AppColors.border, child:
```

```
SizedBox(height: 1)),
            children[i],
          ],
        ],
      ),
    );
  }
}

/// Filtro em pílula (44px). Selecionado = fundo escuro.
class PillChip extends StatelessWidget {
  const PillChip({super.key, required this.label, this.selected = false,
this.onTap});

  final String label;
  final bool selected;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: selected ? AppColors.ink : AppColors.white,
      shape: StadiumBorder(
        side: selected
            ? const BorderSide(color: AppColors.ink, width: 1.5)
            : const BorderSide(color: AppColors.inputBorder),
      ),
      clipBehavior: Clip.antiAlias,
      child: InkWell(
        onTap: onTap,
        child: Container(
          height: 44,
          padding: const EdgeInsets.symmetric(horizontal: 16),
          alignment: Alignment.center,
          child: Text(
            label,
            style: AppText.body(
              14,
              weight: selected ? FontWeight.w700 : FontWeight.w600,
              color: selected ? AppColors.white : AppColors.ink,
            ),
          ),
        ),
      ),
    );
  }
```

```
}

/// Opção de escolha (nível / formato). Selecionado = lima com contorno de
2px.
class ChoiceTile extends StatelessWidget {
  const ChoiceTile({super.key, required this.label, required this.selected,
required this.onTap});

  final String label;
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: selected ? AppColors.lime : AppColors.white,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
        side: selected
            ? const BorderSide(color: AppColors.ink, width: 2)
            : const BorderSide(color: AppColors.inputBorder),
      ),
      clipBehavior: Clip.antiAlias,
      child: InkWell(
        onTap: onTap,
        child: SizedBox(
          height: 48,
          child: Center(
            child: Text(
              label,
              style: AppText.body(14, weight: selected ? FontWeight.w700 :
FontWeight.w600),
            ),
          ),
        ),
      ),
    );
  }
}

/// Barra de progresso fina (6px).
class ThinBar extends StatelessWidget {
  const ThinBar({super.key, required this.value, required this.color,
required this.track});

  final double value;
```

```
  final Color color;
  final Color track;

  @override
  Widget build(BuildContext context) {
    return ClipRRect(
      borderRadius: BorderRadius.circular(3),
      child: SizedBox(
        height: 6,
        child: Stack(
          fit: StackFit.expand,
          children: [
            ColoredBox(color: track),
            FractionallySizedBox(
              alignment: Alignment.centerLeft,
              widthFactor: value.clamp(0, 1).toDouble(),
              child: DecoratedBox(
                decoration: BoxDecoration(color: color, borderRadius:
BorderRadius.circular(3)),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

3.6 `lib/widgets/bottom_nav.dart`

A barra de baixo tem 76 px de altura mais a margem de segurança do telemóvel (a zona do gesto de “voltar ao início”). Tem 5 entradas:

Índice	Entrada	Acção
0	Jogadores	Muda de separador
1	Mapa	Muda de separador
2	Botão verde +	Abre o ecrã Criar jogo (não é um separador)
3	Convites	Muda de separador
4	Perfil	Muda de separador

A entrada ativa fica verde (`primary`), com o traço do ícone a 2,2 em vez de 1,8 e o texto a 700. A lista `_items` usa *records* do Dart 3: `(AppIcon.users, 'Jogadores')` é um par ícone + texto, e o `null` marca o botão +.

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import 'app_icons.dart';

/// Navegação principal: Jogadores, Mapa, (+) Criar jogo, Convites, Perfil.
class AppBottomNav extends StatelessWidget {
  const AppBottomNav({super.key, required this.currentIndex, required
this.onTap});

  static const createIndex = 2;

  final int currentIndex;
  final ValueChanged<int> onTap;

  static const _items = <(AppIcon, String)?>[
    (AppIcon.users, 'Jogadores'),
    (AppIcon.pin, 'Mapa'),
    null,
    (AppIcon.bell, 'Convites'),
    (AppIcon.user, 'Perfil'),
  ];

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: EdgeInsets.only(bottom: MediaQuery.paddingOf(context).bottom),
      decoration: const BoxDecoration(
        color: AppColors.white,
        border: Border(top: BorderSide(color: AppColors.border)),
      ),
      child: SizedBox(
        height: 76,
        child: Row(
          children: [
            for (var i = 0; i < _items.length; i++)
              Expanded(child: _buildItem(i, _items[i])),
          ],
        ),
      ),
    );
```

```
  }

  Widget _buildItem(int index, (AppIcon, String)? item) {
    if (item == null) {
      return Semantics(
        button: true,
        label: 'Criar jogo',
        child: InkResponse(
          onTap: () => onTap(index),
          radius: 32,
          child: Center(
            child: Container(
              width: 52,
              height: 52,
              decoration: const BoxDecoration(color: AppColors.primary,
shape: BoxShape.circle),
              alignment: Alignment.center,
              child: const SvgIcon(AppIcon.plus, size: 24, color:
AppColors.white, strokeWidth: 2.2),
            ),
          ),
        ),
      );
    }

    final active = index == currentIndex;
    final color = active ? AppColors.primary : AppColors.muted;
    return InkResponse(
      onTap: () => onTap(index),
      radius: 36,
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          SvgIcon(item.$1, size: 22, color: color, strokeWidth: active ? 2.2
: 1.8),
          const SizedBox(height: 4),
          Text(
            item.$2,
            style: AppText.body(11, weight: active ? FontWeight.w700 :
FontWeight.w600, color: color),
          ),
        ],
      ),
    );
  }
}
```

Passo 4 — Dados de exemplo

Os nomes, níveis e horários do mockup ficam todos em `lib/data/mock_data.dart`. Quando tiverem backend (Firebase, uma API…), é só este ficheiro que trocam por dados reais; os ecrãs continuam iguais.

Elemento	O que guarda	Usado em
`Player` + `nearbyPlayers`	Os 3 jogadores (Tchali 91%, Helio 84%, Cristiano 72%)	Encontrar jogadores, Perfil do jogador
`Sport` + `sports`	Os 8 desportos e o subtítulo de cada um	Onboarding, Criar jogo
`compatibilityBreakdown`	Os 5 critérios de compatibilidade (30 + 25 + 16 + 12 + 8 = 91)	Perfil do jogador
`GameParticipant` + `gameParticipants`	Quem vai ao jogo	Detalhe do jogo
`levels`, `formats`	As opções de nível e de formato	Criar jogo

No `Player`, `availableHighlight` pinta a disponibilidade a verde ou a cinzento, e `showActions` mostra ou esconde os botões "Ver perfil" e "Convidar". No mockup, o cartão do Cristiano não tem botões; se quiserem botões em todos os cartões, ponham `showActions: true`.

`lib/data/mock_data.dart`

```
/// Dados de exemplo usados nos ecrãs (iguais ao mockup).
/// Substituir por dados reais quando houver backend.
library;

class Player {
  const Player({
    required this.name,
    required this.sport,
    required this.level,
    required this.distance,
    required this.compatibility,
    required this.availability,
    this.availableHighlight = true,
    this.showActions = true,
  });
```

```
  final String name;
  final String sport;
  final String level;
  final String distance;
  final int compatibility;
  final String availability;
  final bool availableHighlight;
  final bool showActions;

  String get initial => name.substring(0, 1);
  String get summary => '$sport · $level · $distance';
}

const nearbyPlayers = <Player>[
  Player(
    name: 'Tchali',
    sport: 'Padel',
    level: 'Intermédio',
    distance: '1,8 km',
    compatibility: 91,
    availability: 'Disponível hoje 19:00 – 21:00',
  ),
  Player(
    name: 'Helio',
    sport: 'Padel',
    level: 'Intermédio',
    distance: '3,1 km',
    compatibility: 84,
    availability: 'Disponível hoje 18:00 – 20:00',
  ),
  Player(
    name: 'Cristiano',
    sport: 'Padel',
    level: 'Avançado',
    distance: '4,2 km',
    compatibility: 72,
    availability: 'Disponível hoje 20:00 – 22:00',
    availableHighlight: false,
    showActions: false,
  ),
];

class Sport {
  const Sport(this.name, this.subtitle);
```

```
  final String name;
  final String subtitle;
}

const sports = <Sport>[
  Sport('Padel', '2 ou 4 jogadores'),
  Sport('Ténis', '2 ou 4 jogadores'),
  Sport('Futebol', '5, 7 ou 11 por equipa'),
  Sport('BJJ', 'Parceiro de treino'),
  Sport('Basquetebol', '3x3 ou 5x5'),
  Sport('Voleibol', 'Praia ou pavilhão'),
  Sport('Running', 'Companhia de corrida'),
  Sport('Ginásio', 'Parceiro de treino'),
];

/// Pontuação de compatibilidade (ecrã "Perfil do jogador").
const compatibilityBreakdown = <(String, int, int)>[
  ('Mesmo desporto', 30, 30),
  ('Nível', 25, 25),
  ('Disponibilidade', 16, 20),
  ('Distância', 12, 15),
  ('Objetivo', 8, 10),
];

class GameParticipant {
  const GameParticipant(this.name, this.detail, {this.organizer = false});

  final String name;
  final String detail;
  final bool organizer;
}

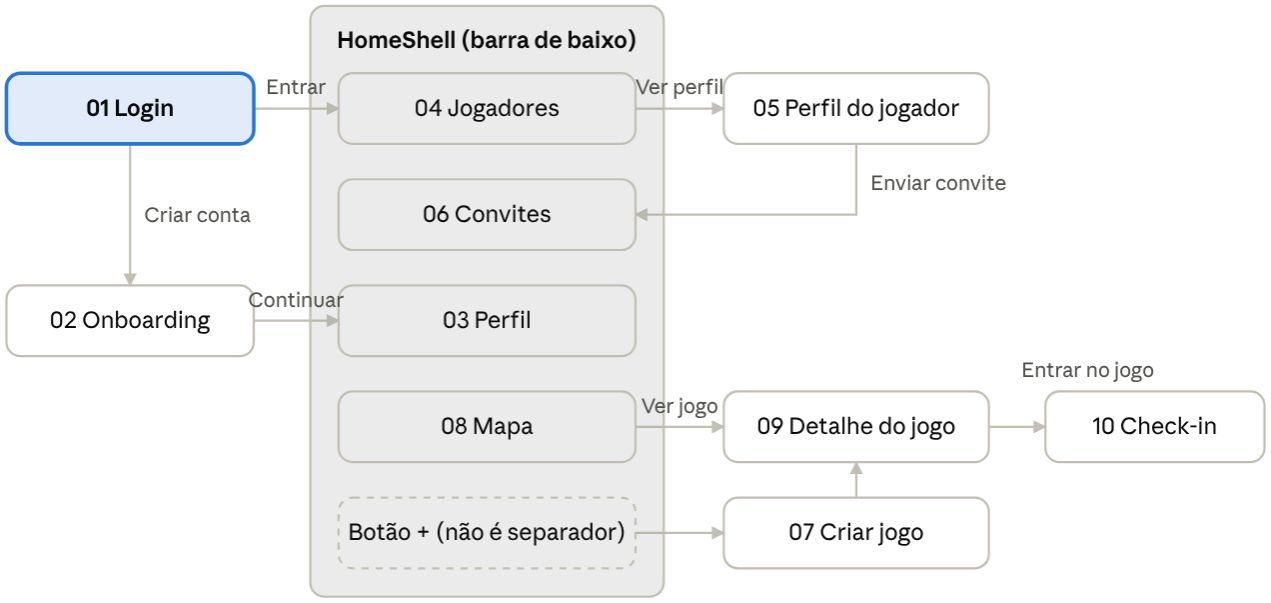
const gameParticipants = <GameParticipant>[
  GameParticipant('Sebastião', 'Organizador', organizer: true),
  GameParticipant('Helio', 'Intermédio'),
  GameParticipant('Cristiano', 'Avançado'),
];

const levels = ['Iniciante', 'Intermédio', 'Avançado'];
const formats = ['Amigável', 'Competitivo'];
```

Passo 5 — Arranque e navegação

A app abre no Login. Depois de entrar, tudo acontece dentro do `HomeShell`, que guarda os 4 separadores e a barra de baixo; os restantes ecrãs abrem por cima dele com `Navigator.push`.

A app abre no Login e tudo parte dos 4 separadores do HomeShell



mapa de navegação · 10 ecrãs

Os 4 separadores ficam dentro do `HomeShell`; os ecrãs da direita abrem por cima deles, e "Enviar convite" volta ao separador Convites.

5.1 Como se navega em Flutter (resumo)

Comando	O que faz	Exemplo na app
`Navigator.push(...)`	Abre um ecrã por cima; o "voltar" regressa ao anterior	Ver perfil, Criar jogo, Ver jogo
`Navigator.pop(context)`	Fecha o ecrã atual	Botões de voltar e fechar
`Navigator.pop(context, true)`	Fecha e devolve um valor a quem o abriu	"Enviar convite" devolve `true`
`pushReplacement(...)`	Troca o ecrã atual por outro (sem voltar atrás)	Login → Jogadores, Criar jogo → Detalhe

Comando	O que faz	Exemplo na app
`pushAndRemoveUntil(..., (_) => false)`	Abre um ecrã e apaga o histórico todo	Onboarding → Perfil

5.2 `lib/main.dart`

Prepara a barra de estado transparente, aplica o tema e define o português de Portugal (`pt_PT`) como língua, para que o calendário e o relógio do ecrã Criar jogo apareçam em português. O primeiro ecrã (`home`) é o `LoginScreen` .

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_localizations/flutter_localizations.dart';

import 'screens/login_screen.dart';
import 'theme/app_theme.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  SystemChrome.setSystemUIOverlayStyle(AppTheme.darkStatusIcons);
  runApp(const WhoPlayApp());
}

class WhoPlayApp extends StatelessWidget {
  const WhoPlayApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'WhoPlay',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light,
      locale: const Locale('pt', 'PT'),
      supportedLocales: const [Locale('pt', 'PT')],
      localizationsDelegates: GlobalMaterialLocalizations.delegates,
      home: const LoginScreen(),
    );
  }
}
```

5.3 `lib/screens/home_shell.dart`

O `HomeShell` é o “esqueleto” com a barra de baixo. Funciona assim:

- **`_tab`** guarda o índice da barra (0, 1, 3 ou 4). As constantes `HomeShell.playersTab`, `mapTab`, `invitesTab` e `profileTab` evitam números soltos no código.
- **`IndexedStack`** mantém os 4 separadores vivos ao mesmo tempo e mostra só um. Assim, quando mudam de separador, os filtros e o scroll não se perdem. Como o botão + (índice 2) não é um separador, o `_stackIndex` converte o índice da barra para a posição no `IndexedStack`.
- **`_onNavTap`**: se carregarem no +, abre o Criar jogo por cima; nos outros casos troca de separador.
- **`onInviteSent`**: quando o Perfil do jogador devolve `true` ("Enviar convite"), o shell salta para o separador Convites.
- O parâmetro **`initialTab`** deixa escolher com que separador abrir; o Onboarding usa-o para abrir no Perfil.

```
import 'package:flutter/material.dart';

import '../theme/app_theme.dart';
import '../widgets/bottom_nav.dart';
import 'create_game_screen.dart';
import 'find_players_screen.dart';
import 'invites_screen.dart';
import 'map_screen.dart';
import 'profile_screen.dart';

/// Estrutura principal com a barra de navegação inferior.
class HomeShell extends StatefulWidget {
  const HomeShell({super.key, this.initialTab = playersTab});

  static const playersTab = 0;
  static const mapTab = 1;
  static const invitesTab = 3;
  static const profileTab = 4;

  final int initialTab;

  @override
  State<HomeShell> createState() => _HomeShellState();
}

class _HomeShellState extends State<HomeShell> {
  late int _tab = widget.initialTab;

  static const _stackIndex = {
    HomeShell.playersTab: 0,
    HomeShell.mapTab: 1,
```

```
    HomeShell.invitesTab: 2,
    HomeShell.profileTab: 3,
  };

  void _onNavTap(int index) {
    if (index == AppBottomNav.createIndex) {
      Navigator.of(context).push(MaterialPageRoute(builder: (_) => const
CreateGameScreen()));
      return;
    }
    setState(() => _tab = index);
  }

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.darkStatusIcons,
      child: Scaffold(
        body: IndexedStack(
          index: _stackIndex[_tab]!,
          children: [
            FindPlayersScreen(onInviteSent: () => setState(() => _tab =
HomeShell.invitesTab)),
            const MapScreen(),
            const InvitesScreen(),
            const ProfileScreen(),
          ],
        ),
        bottomNavigationBar: AppBottomNav(currentIndex: _tab, onTap:
_onNavTap),
      ),
    );
  }
}
```

O `AnnotatedRegion` diz ao sistema que cor devem ter os ícones da barra de estado (relógio, bateria) enquanto este ecrã está visível. Os ecrãs com fundo escuro (Login, Detalhe do jogo, Check-in) usam `lightStatusIcons`.

Ecrã 01 — Login

O ecrã divide-se em 2 zonas: um cabeçalho verde-floresta com o logo e o slogan, e uma zona clara com o formulário. Os botões "Ainda não tens conta?" e "Criar conta" ficam colados ao fundo.

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Cabeçalho	fundo #123D29, padding 88 / 28 / 44	`Container(color: AppColors.forest, padding: ...topInset(context, 88)...)`
Logo	símbolo 60 px, "WhoPlay" 52 px 800	`WhoPlayLogo()` (valores por omissão)
Slogan	19 px, line-height 1.4, max-width 250	`ConstrainedBox(maxWidth: 250)` + `AppText.body(19, height: 1.4)`
Formulário	padding 32 / 28 / 28, espaço de 18 entre elementos	`Padding(...)` + `SizedBox(height: 18)`
Título "Entrar"	28 px 700, letter-spacing -0.5	`AppText.display(28, letterSpacing: -0.5)`
Campos	52 px, borda #B9C4BA, raio 12	`AppTextField`
"Esqueci-me…"	14 px 600, verde, alinhado à direita	`Align(centerRight)` + `InkWell`
"Criar conta"	`margin-top: auto`, contorno 1,5 px	`Spacer()` + `AppButton(outlined: true)`

Comportamento

- **Entrar** usa `pushReplacement` para abrir o `HomeShell` no separador Jogadores. O Login sai do histórico, e o botão "voltar" do Android já não regressa a ele.
- **Criar conta** usa `push` para abrir o Onboarding, de onde se pode voltar.
- **Esqueci-me da palavra-passe** mostra uma mensagem (ainda não há ecrã de recuperação).
- Os campos vêm preenchidos com os valores do mockup. Para os ter vazios, tirem o `initialValue`.

`lib/screens/login_screen.dart`

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
```

```
import '../widgets/fields.dart';
import '../widgets/logo.dart';
import 'home_shell.dart';
import 'onboarding_screen.dart';

/// 01 Login
class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.lightStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Container(
                color: AppColors.forest,
                padding: EdgeInsets.fromLTRB(28, topInset(context, 88), 28,
44),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    const WhoPlayLogo(),
                    const SizedBox(height: 14),
                    ConstrainedBox(
                      constraints: const BoxConstraints(maxWidth: 250),
                      child: Text(
                        'Encontra pessoas. Marca o jogo. Vai jogar.',
                        style: AppText.body(19, color: AppColors.white,
height: 1.4),
                      ),
                    ),
                  ],
                ),
              ),
              Expanded(
                child: Padding(
                  padding: EdgeInsets.fromLTRB(28, 32, 28, 28 +
bottomInset(context)),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.stretch,
                    children: [
                      Text('Entrar', style: AppText.display(28,
```

```
letterSpacing: -0.5)),
                      const SizedBox(height: 18),
                      const AppTextField(
                        label: 'Email',
                        initialValue: 'sebastiao@email.com',
                        keyboardType: TextInputType.emailAddress,
                      ),
                      const SizedBox(height: 18),
                      const AppTextField(
                        label: 'Palavra-passe',
                        initialValue: '12345678',
                        obscureText: true,
                      ),
                      const SizedBox(height: 18),
                      Align(
                        alignment: Alignment.centerRight,
                        child: InkWell(
                          onTap: () => showInfo(context, 'Recuperação de
palavra-passe em breve.'),
                          child: Padding(
                            padding: const EdgeInsets.symmetric(vertical: 6),
                            child: Text(
                              'Esqueci-me da palavra-passe',
                              style: AppText.body(14, weight:
FontWeight.w600, color: AppColors.primary),
                            ),
                          ),
                        ),
                      ),
                      const SizedBox(height: 18),
                      AppButton(
                        label: 'Entrar',
                        onPressed: () =>
Navigator.of(context).pushReplacement(
                          MaterialPageRoute(builder: (_) => const
HomeShell()),
                        ),
                      ),
                      const SizedBox(height: 18),
                      const Spacer(),
                      Text(
                        'Ainda não tens conta?',
                        textAlign: TextAlign.center,
                        style: AppText.body(14, color: AppColors.muted),
                      ),
                      const SizedBox(height: 12),
```

```
                      AppButton(
                        label: 'Criar conta',
                        outlined: true,
                        onPressed: () => Navigator.of(context).push(
                          MaterialPageRoute(builder: (_) => const
OnboardingScreen()),
                        ),
                      ),
                    ],
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Ecrã 02 — Onboarding (desportos)

É o passo 1 de 3 do registo: escolher um ou mais desportos numa grelha de 2 colunas. Este é o primeiro ecrã com estado (*state*): os desportos escolhidos mudam quando se toca, por isso é um `StatefulWidget`.

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Ecrã	padding 56 / 24 / 28	`FillScrollView(padding: ...topInset(context, 56)...)`
Barra de progresso	3 traços de 6 px, espaço 6, raio 3	`_StepProgress(step: 1, total: 3)`
Título	30 px 700, line-height 1.12, margem 28 em cima e 8 em baixo	`AppText.display(30, letterSpacing: -0.6, height: 1.12)`
Grelha	2 colunas, espaço 12, cartões de 88 px	Linhas de 2 com `Expanded` e `SizedBox(width: 12)`
Cartão selecionado	fundo lima, borda 2 px #10201A, visto de 20 px	`_SportTile(selected: true)`

Elemento	Medidas do mockup	No código
Cartão normal	fundo branco, borda 1 px #B9C4BA, subtítulo cinzento	`_SportTile(selected: false)`
Rodapé	texto 14 px centrado + botão "Continuar"	`Spacer()` + `Text` + `AppButton`

Comportamento

- `_selected` é um `Set` com os nomes escolhidos e começa com Padel e Ténis, como no mockup. `_toggle` tira o desporto se já lá estiver e põe-no se não estiver; o `setState` redesenha o ecrã.
- O texto do rodapé conta os desportos e acerta o singular: "1 desporto escolhido" ou "2 desportos escolhidos".
- **Continuar** fica desativado (`onPressed: null`) quando não há nenhum desporto escolhido. Quando está ativo, abre o `HomeShell` no Perfil e apaga o histórico (`pushAndRemoveUntil`).
- A grelha é feita com um `for` que avança de 2 em 2 (`i += 2`), criando uma `Row` por par de desportos. Com uma quantidade ímpar, o último lugar fica vazio (`SizedBox()`).
- Os passos 2 e 3 (nível e disponibilidade) ainda não existem no mockup.

`lib/screens/onboarding_screen.dart`

```
import 'package:flutter/material.dart';

import '../data/mock_data.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import 'home_shell.dart';

/// 02 Onboarding — desportos (passo 1 de 3)
class OnboardingScreen extends StatefulWidget {
  const OnboardingScreen({super.key});

  @override
  State<OnboardingScreen> createState() => _OnboardingScreenState();
}

class _OnboardingScreenState extends State<OnboardingScreen> {
  final _selected = <String>{'Padel', 'Ténis'};
```

```
  void _toggle(String sport) => setState(() {
        if (!_selected.remove(sport)) _selected.add(sport);
      });

  @override
  Widget build(BuildContext context) {
    final count = _selected.length;
    final countLabel = count == 1 ? '1 desporto escolhido' : '$count
desportos escolhidos';

    return AnnotatedRegion(
      value: AppTheme.darkStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          padding: EdgeInsets.fromLTRB(24, topInset(context, 56), 24, 28 +
bottomInset(context)),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const _StepProgress(step: 1, total: 3),
              const SizedBox(height: 28),
              Text(
                'Que desportos queres jogar?',
                style: AppText.display(30, letterSpacing: -0.6, height:
1.12),
              ),
              const SizedBox(height: 8),
              Text(
                'Escolhe um ou mais. Podes mudar isto no teu perfil.',
                style: AppText.body(16, color: AppColors.muted, height:
1.45),
              ),
              const SizedBox(height: 24),
              for (var i = 0; i < sports.length; i += 2) ...[
                if (i > 0) const SizedBox(height: 12),
                Row(
                  children: [
                    Expanded(child: _tile(sports[i])),
                    const SizedBox(width: 12),
                    Expanded(child: i + 1 < sports.length ? _tile(sports[i +
1]) : const SizedBox()),
                  ],
                ),
              ],
              const SizedBox(height: 24),
```

```
              const Spacer(),
              Text(
                '$countLabel · a seguir: nível e disponibilidade',
                textAlign: TextAlign.center,
                style: AppText.body(14, color: AppColors.muted),
              ),
              const SizedBox(height: 10),
              AppButton(
                label: 'Continuar',
                onPressed: count == 0
                    ? null
                    : () => Navigator.of(context).pushAndRemoveUntil(
                          MaterialPageRoute(builder: (_) => const
HomeShell(initialTab: HomeShell.profileTab)),
                          (_) => false,
                        ),
              ),
            ],
          ),
        ),
      ),
    );
  }

  Widget _tile(Sport sport) => _SportTile(
        sport: sport,
        selected: _selected.contains(sport.name),
        onTap: () => _toggle(sport.name),
      );
}

class _StepProgress extends StatelessWidget {
  const _StepProgress({required this.step, required this.total});

  final int step;
  final int total;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Expanded(
          child: Row(
            children: [
              for (var i = 0; i < total; i++) ...[
                if (i > 0) const SizedBox(width: 6),
```

```
                Expanded(
                  child: Container(
                    height: 6,
                    decoration: BoxDecoration(
                      color: i < step ? AppColors.primary : AppColors.track,
                      borderRadius: BorderRadius.circular(3),
                    ),
                  ),
                ),
              ],
            ],
          ),
        ),
        const SizedBox(width: 12),
        Text(
          'Passo $step de $total',
          style: AppText.body(13, weight: FontWeight.w600, color:
AppColors.muted),
        ),
      ],
    );
  }
}

class _SportTile extends StatelessWidget {
  const _SportTile({required this.sport, required this.selected, required
this.onTap});

  final Sport sport;
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Semantics(
      selected: selected,
      button: true,
      child: Material(
        color: selected ? AppColors.lime : AppColors.white,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(16),
          side: selected
              ? const BorderSide(color: AppColors.ink, width: 2)
              : const BorderSide(color: AppColors.inputBorder),
        ),
        clipBehavior: Clip.antiAlias,
```

```
        child: InkWell(
          onTap: onTap,
          child: Container(
            height: 88,
            padding: const EdgeInsets.all(14),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Row(
                  children: [
                    Expanded(
                      child: Text(
                        sport.name,
                        overflow: TextOverflow.ellipsis,
                        style: AppText.display(19),
                      ),
                    ),
                    if (selected)
                      const SvgIcon(AppIcon.check, size: 20, color:
AppColors.ink, strokeWidth: 2.4),
                  ],
                ),
                Text(
                  sport.subtitle,
                  style: AppText.body(
                    13,
                    weight: FontWeight.w500,
                    color: selected ? AppColors.ink : AppColors.muted,
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

Ecrã 03 — Perfil

O perfil do próprio utilizador: um cartão verde com o nome e a zona, seguido de 3 secções em cartões brancos. Não tem estado, por isso é um `StatelessWidget`. Vive dentro do `HomeShell`, que já desenha a barra de baixo; por isso o ecrã não tem `Scaffold` próprio e

devolve diretamente um `SingleChildScrollView`.

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Ecrã	padding 52 / 20, espaço 18 entre blocos	`topInset(context, 52)` + `SizedBox(height: 18)`
Cabeçalho	título 28 px + pílula "Editar" 44 px	`Row(spaceBetween)` com `ScreenTitle` e `AppButton.pill`
Cartão verde	#123D29, raio 20, padding 18	`Container(decoration: ...forest...)`
Avatar	64 px, lima com letra escura 28 px 800	`Avatar('S', size: 64, background: lime, foreground: ink)`
Nome	22 px 700 branco	`AppText.display(22, color: white)`
Localização	ícone pin 16 px + 14 px	`SvgIcon(AppIcon.pin, size: 16)`
Desportos e nível	linhas com padding 16, etiqueta raio 14	`_SportLevelRow`, lima se `highlight`
Disponibilidade	linhas com padding 14 / 16	`_AvailabilityRow`
Objetivo	cartão único, 16 px 600	`SurfaceCard`

Comportamento

- **Editar** mostra uma mensagem; o ecrã de edição não está no mockup.
- Os separadores entre as linhas de cada cartão vêm do `ListCard`, que põe um traço de 1 px antes de cada linha a partir da segunda.
- `Flexible` à volta do texto da localização deixa-o passar para a linha seguinte em vez de transbordar quando o nome da zona é comprido.

`lib/screens/profile_screen.dart`

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';

/// 03 Perfil
class ProfileScreen extends StatelessWidget {
```

```
  const ProfileScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
      padding: EdgeInsets.fromLTRB(20, topInset(context, 52), 20, 20),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              const ScreenTitle('Perfil'),
              AppButton.pill(
                label: 'Editar',
                onPressed: () => showInfo(context, 'Edição do perfil em
breve.'),
              ),
            ],
          ),
          const SizedBox(height: 18),
          Container(
            padding: const EdgeInsets.all(18),
            decoration: BoxDecoration(color: AppColors.forest, borderRadius:
BorderRadius.circular(20)),
            child: Row(
              children: [
                const Avatar('S', size: 64, fontSize: 28, background:
AppColors.lime, foreground: AppColors.ink),
                const SizedBox(width: 16),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text('Sebastião', style: AppText.display(22, color:
AppColors.white)),
                      const SizedBox(height: 4),
                      Row(
                        children: [
                          const SvgIcon(AppIcon.pin, size: 16, color:
AppColors.white),
                          const SizedBox(width: 6),
                          Flexible(
                            child: Text(
                              'Moscavide · raio até 5 km',
                              style: AppText.body(14, color:
```

```
AppColors.white),
                              ),
                            ),
                          ],
                        ),
                      ],
                    ),
                  ),
                ],
              ),
            ),
            const SizedBox(height: 18),
            const Section(
              title: 'Desportos e nível',
              child: ListCard(
                children: [
                  _SportLevelRow('Padel', 'Intermédio', highlight: true),
                  _SportLevelRow('Ténis', 'Iniciante'),
                ],
              ),
            ),
            const SizedBox(height: 18),
            const Section(
              title: 'Disponibilidade',
              child: ListCard(
                children: [
                  _AvailabilityRow('Segunda', '18h – 21h'),
                  _AvailabilityRow('Quarta', '18h – 22h'),
                  _AvailabilityRow('Sábado', '10h – 15h'),
                ],
              ),
            ),
            const SizedBox(height: 18),
            Section(
              title: 'Objetivo',
              child: SurfaceCard(
                padding: const EdgeInsets.symmetric(horizontal: 16, vertical:
14),
                child: Text('Jogos amigáveis', style: AppText.body(16, weight:
FontWeight.w600)),
              ),
            ),
          ],
        ),
      );
  }
```

```
}

class _SportLevelRow extends StatelessWidget {
  const _SportLevelRow(this.sport, this.level, {this.highlight = false});

  final String sport;
  final String level;
  final bool highlight;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(16),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Text(sport, style: AppText.body(17, weight: FontWeight.w700)),
          Tag(
            level,
            background: highlight ? AppColors.lime : AppColors.chip,
            radius: 14,
            padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 5),
          ),
        ],
      ),
    );
  }
}

class _AvailabilityRow extends StatelessWidget {
  const _AvailabilityRow(this.day, this.hours);

  final String day;
  final String hours;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Text(day, style: AppText.body(16, weight: FontWeight.w600)),
          Text(hours, style: AppText.body(16, color: AppColors.muted)),
        ],
```

```
      ),
    );
  }
}
```

Ecrã 04 — Encontrar jogadores

É o separador principal depois do login: filtros em pílula no topo e uma lista de cartões de jogadores, cada um com a compatibilidade em percentagem. O cartão está num widget público (`PlayerCard`) para poder ser reutilizado noutros ecrãs.

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Ecrã	padding 52 / 20, espaço 14	`topInset(context, 52)` + `SizedBox(height: 14)`
Filtros	pílulas de 44 px, espaço 8	`PillChip` numa `Row` com scroll horizontal
Contador	14 px cinzento	`'${nearbyPlayers.length} jogadores compatíveis perto de ti'`
Cartão	branco, raio 18, padding 14, espaço 12	`SurfaceCard(radius: 18)`
Avatar	48 px, verde-floresta com letra lima 20 px	`Avatar(player.initial)`
Compatibilidade	Bricolage 18 px 800, raio 12, padding 6 / 10	`Tag(display: true, fontSize: 18)`
Cor da percentagem	lima se ≥ 80%, cinzenta se menor	`player.compatibility >= 80 ? lime : chip`
Disponibilidade	relógio 16 px + 14 px 600, verde ou cinzento	`availableHighlight` decide a cor
Botões	2 x 44 px, espaço 8	`AppButton.small` dentro de `Expanded`

Comportamento

- **Filtros:** Padel e Ténis são de escolha única (`_sport`); "Até 5 km" e "Hoje" ligam e desligam. Por agora só mudam o aspeto: a lista não é filtrada, porque os dados de exemplo são todos de Padel.

- O `SingleChildScrollView` horizontal à volta dos filtros evita que as pílulas transbordem em ecrãs estreitos.
- **Ver perfil** abre o Perfil do jogador com `push<bool>` e espera pela resposta (`await`). Se a resposta for `true` ("Enviar convite"), chama `widget.onInviteSent`, e o `HomeShell` salta para Convites.
- **Convidar** mostra uma mensagem de confirmação.

`lib/screens/find_players_screen.dart`

```
import 'package:flutter/material.dart';

import '../data/mock_data.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import 'player_profile_screen.dart';

/// 04 Encontrar jogadores
class FindPlayersScreen extends StatefulWidget {
  const FindPlayersScreen({super.key, this.onInviteSent});

  final VoidCallback? onInviteSent;

  @override
  State<FindPlayersScreen> createState() => _FindPlayersScreenState();
}

class _FindPlayersScreenState extends State<FindPlayersScreen> {
  String _sport = 'Padel';
  bool _within5km = false;
  bool _today = false;

  Future<void> _openProfile(Player player) async {
    final invited = await Navigator.of(context).push<bool>(
      MaterialPageRoute(builder: (_) => PlayerProfileScreen(player: player)),
    );
    if (invited == true) widget.onInviteSent?.call();
  }

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
      padding: EdgeInsets.fromLTRB(20, topInset(context, 52), 20, 20),
```

```
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const ScreenTitle('Encontrar jogadores'),
          const SizedBox(height: 14),
          SingleChildScrollView(
            scrollDirection: Axis.horizontal,
            clipBehavior: Clip.none,
            child: Row(
              children: [
                for (final s in ['Padel', 'Ténis']) ...[
                  PillChip(label: s, selected: _sport == s, onTap: () =>
setState(() => _sport = s)),
                  const SizedBox(width: 8),
                ],
                PillChip(
                  label: 'Até 5 km',
                  selected: _within5km,
                  onTap: () => setState(() => _within5km = !_within5km),
                ),
                const SizedBox(width: 8),
                PillChip(label: 'Hoje', selected: _today, onTap: () =>
setState(() => _today = !_today)),
              ],
            ),
          ),
          const SizedBox(height: 14),
          Text(
            '${nearbyPlayers.length} jogadores compatíveis perto de ti',
            style: AppText.body(14, color: AppColors.muted),
          ),
          const SizedBox(height: 14),
          for (var i = 0; i < nearbyPlayers.length; i++) ...[
            if (i > 0) const SizedBox(height: 12),
            PlayerCard(
              player: nearbyPlayers[i],
              onViewProfile: () => _openProfile(nearbyPlayers[i]),
              onInvite: () => showInfo(context, 'Convite enviado a
${nearbyPlayers[i].name}.'),
            ),
          ],
        ],
      ),
    );
  }
}
```

```
class PlayerCard extends StatelessWidget {
  const PlayerCard({super.key, required this.player, this.onViewProfile,
this.onInvite});

  final Player player;
  final VoidCallback? onViewProfile;
  final VoidCallback? onInvite;

  @override
  Widget build(BuildContext context) {
    final availabilityColor = player.availableHighlight ? AppColors.primary :
AppColors.muted;
    return SurfaceCard(
      radius: 18,
      padding: const EdgeInsets.all(14),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Row(
            children: [
              Avatar(player.initial),
              const SizedBox(width: 12),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(player.name, style: AppText.body(17, weight:
FontWeight.w700)),
                    const SizedBox(height: 2),
                    Text(player.summary, style: AppText.body(14, color:
AppColors.muted)),
                  ],
                ),
              ),
              const SizedBox(width: 12),
              Tag(
                '${player.compatibility}%',
                display: true,
                fontSize: 18,
                background: player.compatibility >= 80 ? AppColors.lime :
AppColors.chip,
              ),
            ],
          ),
          const SizedBox(height: 12),
```

```
          Row(
            children: [
              SvgIcon(AppIcon.clock, size: 16, color: availabilityColor),
              const SizedBox(width: 6),
              Text(
                player.availability,
                style: AppText.body(14, weight: FontWeight.w600, color:
availabilityColor),
              ),
            ],
          ),
          if (player.showActions) ...[
            const SizedBox(height: 12),
            Row(
              children: [
                Expanded(child: AppButton.small(label: 'Ver perfil',
outlined: true, onPressed: onViewProfile)),
                const SizedBox(width: 8),
                Expanded(child: AppButton.small(label: 'Convidar', onPressed:
onInvite)),
              ],
            ),
          ],
        ],
      ),
    );
  }
}
```

Ecrã 05 — Perfil do jogador

Mostra o perfil de outro jogador e explica a compatibilidade (91% para o Tchali), critério a critério, com barras. O ecrã recebe o `Player` no construtor, por isso serve para qualquer jogador da lista.

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Ecrã	padding 48 / 20 / 28, espaço 16	`FillScrollView` + `SizedBox(height: 16)`
Voltar	círculo branco de 44 px com borda	`CircleIconButton` + `SvgIcon(AppIcon.back)`

Elemento	Medidas do mockup	No código
Avatar	72 px, letra 30 px	`Avatar(player.initial, size: 72, fontSize: 30)`
Avaliação	estrela preenchida 16 px + "4,7 de avaliação" 15 px 700	`SvgIcon(AppIcon.star, strokeWidth: 1.5)`
Cartão de compatibilidade	#123D29, raio 20, padding 20, espaço 14	`_CompatibilityCard`
Percentagem	Bricolage 44 px 800 lima, alinhada pela linha de base	`crossAxisAlignment: baseline`
Barras	6 px, calha #2C5A43, enchimento lima	`ThinBar(value: score / max)`
Informações	2 linhas, 15 px, rótulo cinzento e valor 700	`ListCard` + `_InfoRow`
Botão	"Enviar convite" colado ao fundo	`Spacer()` + `AppButton`

Comportamento

- Os 5 critérios vêm de `compatibilityBreakdown`. O `for (final (label, score, max) in ...)` desmonta cada *record* nas suas 3 partes (padrões do Dart 3).
- `availability.replaceFirst('Disponível hoje ', '')` aproveita o texto do cartão da lista e mostra só o horário ("19:00 – 21:00").
- **Enviar convite** fecha o ecrã com `Navigator.pop(true)`. Esse `true` chega ao ecrã Encontrar jogadores, que muda para o separador Convites.
- No `_InfoRow`, o valor está num `Expanded` com `textAlign: TextAlign.end`, para que um texto comprido ("Procura parceiro para jogo") quebre de linha em vez de transbordar.

`lib/screens/player_profile_screen.dart`

```
import 'package:flutter/material.dart';

import '../data/mock_data.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
```

```
/// 05 Perfil do jogador. Devolve `true` ao fazer pop se o convite foi
enviado.
class PlayerProfileScreen extends StatelessWidget {
  const PlayerProfileScreen({super.key, required this.player});

  final Player player;

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.darkStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          padding: EdgeInsets.fromLTRB(20, topInset(context, 48), 20, 28 +
bottomInset(context)),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Align(
                alignment: Alignment.centerLeft,
                child: CircleIconButton(
                  semanticLabel: 'Voltar',
                  onPressed: () => Navigator.of(context).pop(),
                  child: const SvgIcon(AppIcon.back),
                ),
              ),
              const SizedBox(height: 16),
              Row(
                children: [
                  Avatar(player.initial, size: 72, fontSize: 30),
                  const SizedBox(width: 16),
                  Expanded(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(player.name, style: AppText.display(28,
letterSpacing: -0.5)),
                        const SizedBox(height: 4),
                        Text(player.summary, style: AppText.body(15, color:
AppColors.muted)),
                        const SizedBox(height: 4),
                        Row(
                          children: [
                            const SvgIcon(AppIcon.star, size: 16,
strokeWidth: 1.5),
                            const SizedBox(width: 5),
```

```
                            Text('4,7 de avaliação', style: AppText.body(15,
weight: FontWeight.w700)),
                          ],
                        ),
                      ],
                    ),
                  ),
                ],
              ),
              const SizedBox(height: 16),
              _CompatibilityCard(total: player.compatibility),
              const SizedBox(height: 16),
              ListCard(
                children: [
                  _InfoRow('Disponível hoje',
player.availability.replaceFirst('Disponível hoje ', '')),
                  const _InfoRow('Objetivo', 'Procura parceiro para jogo'),
                ],
              ),
              const SizedBox(height: 16),
              const Spacer(),
              AppButton(
                label: 'Enviar convite',
                onPressed: () => Navigator.of(context).pop(true),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class _CompatibilityCard extends StatelessWidget {
  const _CompatibilityCard({required this.total});

  final int total;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(color: AppColors.forest, borderRadius:
BorderRadius.circular(20)),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
```

```
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            crossAxisAlignment: CrossAxisAlignment.baseline,
            textBaseline: TextBaseline.alphabetic,
            children: [
              Text(
                'Compatibilidade contigo',
                style: AppText.body(15, weight: FontWeight.w600, color:
AppColors.white),
              ),
              Text(
                '$total%',
                style: AppText.display(44, weight: FontWeight.w800, color:
AppColors.lime, height: 1),
              ),
            ],
          ),
          for (final (label, score, max) in compatibilityBreakdown) ...[
            const SizedBox(height: 14),
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Text(label, style: AppText.body(14, color: AppColors.white)),
                Text(
                  '$score / $max',
                  style: AppText.body(14, weight: FontWeight.w700, color:
AppColors.white),
                ),
              ],
            ),
            const SizedBox(height: 6),
            ThinBar(value: score / max, color: AppColors.lime, track:
AppColors.forestLight),
          ],
        ],
      ),
    );
  }
}

class _InfoRow extends StatelessWidget {
  const _InfoRow(this.label, this.value);

  final String label;
  final String value;
```

```
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      child: Row(
        children: [
          Text(label, style: AppText.body(15, color: AppColors.muted)),
          const SizedBox(width: 12),
          Expanded(
            child: Text(
              value,
              textAlign: TextAlign.end,
              style: AppText.body(15, weight: FontWeight.w700),
            ),
          ),
        ],
      ),
    );
  }
}
```

Ecrã 06 — Convites e avisos

O ecrã tem 3 secções: o convite enviado (à espera de resposta), o convite recebido (com Recusar e Aceitar) e uma lista de avisos. Os avisos por ler têm um ponto verde; os lidos têm um ponto cinzento.

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Ecrã	padding 52 / 20, espaço 16	`topInset(context, 52)` + `SizedBox(height: 16)`
Cartões	branco, raio 18, padding 14	`SurfaceCard(radius: 18, padding: EdgeInsets.all(14))`
Estado "A aguardar"	13 px 700, fundo #E6ECE3	`Tag('A aguardar', fontSize: 13)`
Convite recebido	título 16 px 700, detalhe 14 px cinzento	`_TitleSubtitle`
Botões	Recusar (contorno) e Aceitar (verde), 44 px	`AppButton.small`

Elemento	Medidas do mockup	No código
Avisos	linhas com padding 14 / 16, ponto de 10 px	`_NoticeRow`
Aviso por ler	ponto #1F7A4D, título 700	`unread: true` (valor por omissão)
Aviso lido	ponto #CDD6CB, título 600	`unread: false`

Comportamento

- `_receivedAnswer` começa a `null`, e enquanto for `null` aparecem os 2 botões. Depois de Aceitar ou Recusar, os botões dão lugar a um texto verde ("Aceitaste o convite" ou "Recusaste o convite") e aparece uma mensagem em baixo.
- `if (...) Row(...) else SizedBox(...)` dentro da lista `children` é a forma do Flutter de mostrar uma coisa ou outra conforme o estado.
- As partes que nunca mudam estão marcadas com `const`, e o Flutter não as volta a construir quando o estado muda.

`lib/screens/invites_screen.dart`

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';

/// 06 Convites e avisos
class InvitesScreen extends StatefulWidget {
  const InvitesScreen({super.key});

  @override
  State<InvitesScreen> createState() => _InvitesScreenState();
}

class _InvitesScreenState extends State<InvitesScreen> {
  String? _receivedAnswer;

  void _answer(bool accepted) {
    setState(() => _receivedAnswer = accepted ? 'Aceitaste o convite' :
'Recusaste o convite');
    showInfo(context, accepted ? 'Convite aceite.' : 'Convite recusado.');
  }
```

```
  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
      padding: EdgeInsets.fromLTRB(20, topInset(context, 52), 20, 20),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const ScreenTitle('Convites'),
          const SizedBox(height: 16),
          const Section(
            title: 'Enviados',
            child: SurfaceCard(
              radius: 18,
              padding: EdgeInsets.all(14),
              child: Row(
                children: [
                  Avatar('T'),
                  SizedBox(width: 12),
                  Expanded(
                    child: _TitleSubtitle(title: 'Tchali', subtitle: 'Padel ·
hoje às 19:00'),
                  ),
                  SizedBox(width: 12),
                  Tag('A aguardar', fontSize: 13),
                ],
              ),
            ),
          ),
          const SizedBox(height: 16),
          Section(
            title: 'Recebidos',
            child: SurfaceCard(
              radius: 18,
              padding: const EdgeInsets.all(14),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  const Row(
                    children: [
                      Avatar('C'),
                      SizedBox(width: 12),
                      Expanded(
                        child: _TitleSubtitle(
                          title: 'Cristiano convidou-te para jogar',
                          subtitle: 'Padel · quarta às 18:00 · 4,2 km',
                        ),
```

```
                    ),
                  ],
                ),
                const SizedBox(height: 12),
                if (_receivedAnswer == null)
                  Row(
                    children: [
                      Expanded(
                        child: AppButton.small(label: 'Recusar', outlined:
true, onPressed: () => _answer(false)),
                      ),
                      const SizedBox(width: 8),
                      Expanded(child: AppButton.small(label: 'Aceitar',
onPressed: () => _answer(true))),
                    ],
                  )
                else
                  SizedBox(
                    height: 44,
                    child: Center(
                      child: Text(
                        _receivedAnswer!,
                        style: AppText.body(15, weight: FontWeight.w700,
color: AppColors.primary),
                      ),
                    ),
                  ),
              ],
            ),
          ),
        ),
        const SizedBox(height: 16),
        const Section(
          title: 'Avisos',
          child: ListCard(
            radius: 18,
            children: [
              _NoticeRow('O Helio aceitou o teu convite', 'Há 5 minutos'),
              _NoticeRow('Falta 1 jogador para o teu jogo', 'Padel · sábado
às 10:00'),
              _NoticeRow('Encontrámos 3 jogadores compatíveis perto de ti',
'Ontem', unread: false),
            ],
          ),
        ),
      ],
```

```
      ),
    );
  }
}

class _TitleSubtitle extends StatelessWidget {
  const _TitleSubtitle({required this.title, required this.subtitle});

  final String title;
  final String subtitle;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(title, style: AppText.body(16, weight: FontWeight.w700)),
        const SizedBox(height: 2),
        Text(subtitle, style: AppText.body(14, color: AppColors.muted)),
      ],
    );
  }
}

class _NoticeRow extends StatelessWidget {
  const _NoticeRow(this.title, this.subtitle, {this.unread = true});

  final String title;
  final String subtitle;
  final bool unread;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      child: Row(
        children: [
          Container(
            width: 10,
            height: 10,
            decoration: BoxDecoration(
              color: unread ? AppColors.primary : AppColors.track,
              shape: BoxShape.circle,
            ),
          ),
          const SizedBox(width: 12),
```

```
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  title,
                  style: AppText.body(15, weight: unread ? FontWeight.w700 :
FontWeight.w600),
                ),
                const SizedBox(height: 2),
                Text(subtitle, style: AppText.body(13, color:
AppColors.muted)),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

Ecrã 07 — Criar jogo

Um formulário para marcar um jogo: desporto, data, hora, local, nível, formato, vagas e preço. Abre-se no botão + da barra de baixo e é o ecrã com mais estado da app.

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Ecrã	padding 48 / 20 / 28, espaço 16	`FillScrollView` + `SizedBox(height: 16)`
Cabeçalho	voltar 44 px + título 28 px, espaço 12	`Row` com `CircleIconButton` e `ScreenTitle`
Desporto	`<select>` de 52 px, 16 px 600	`AppDropdown<String>`
Data / Hora	lado a lado em proporção 3 : 2	`Expanded(flex: 3)` e `Expanded(flex: 2)`
Nível	3 botões de 48 px, espaço 8	`_ChoiceGroup` com `ChoiceTile`
Formato	2 botões de 48 px	`_ChoiceGroup`
Vagas abertas	caixa de 52 px com − / + de 44 px (fundo #E6ECE3, raio 10)	`FieldBox` + `_StepperButton`

Elemento	Medidas do mockup	No código
Preço por pessoa	campo "8 €"	`AppTextField`
Botão	"Criar jogo" colado ao fundo	`Spacer()` + `AppButton`

Comportamento

- **Data e Hora** são campos só de leitura (`readOnly: true`). Tocar neles abre o calendário (`showDatePicker`) ou o relógio (`showTimePicker`), em português graças ao `main.dart` . A data escolhida é escrita no formato do mockup ("Sáb, 3 de outubro") pelo `_formatDate` , que usa 2 listas com os nomes dos dias e dos meses. Assim não é preciso o pacote `intl` .
- Os campos de data e hora usam um `TextEditingController` porque o texto muda pelo código. Os controllers têm de ser libertados no `dispose()` .
- **Vagas abertas** vai de 1 a 3 (capacidade 4 menos o organizador). Nos limites o botão fica desativado e o sinal fica cinzento.
- **Nível e Formato** usam o `_ChoiceGroup` , que desenha uma linha de `ChoiceTile` a partir de uma lista de opções; só uma pode estar selecionada.
- **Criar jogo** abre o Detalhe do jogo com `pushReplacement` ; ao voltar atrás no detalhe, regressa-se aos separadores e não ao formulário. Por agora os valores escolhidos não são enviados: o detalhe mostra o jogo de exemplo, que é o mesmo do mockup.

`lib/screens/create_game_screen.dart`

```
import 'package:flutter/material.dart';

import '../data/mock_data.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import '../widgets/fields.dart';
import 'game_detail_screen.dart';

/// 07 Criar jogo
class CreateGameScreen extends StatefulWidget {
  const CreateGameScreen({super.key});

  @override
  State<CreateGameScreen> createState() => _CreateGameScreenState();
```

```
}

class _CreateGameScreenState extends State<CreateGameScreen> {
  static const _weekdays = ['Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb', 'Dom'];
  static const _months = [
    'janeiro', 'fevereiro', 'março', 'abril', 'maio', 'junho',
    'julho', 'agosto', 'setembro', 'outubro', 'novembro', 'dezembro',
  ];
  static const _capacity = 4;

  String _sport = 'Padel';
  DateTime _date = DateTime(2026, 10, 3);
  TimeOfDay _time = const TimeOfDay(hour: 10, minute: 0);
  String _level = 'Intermédio';
  String _format = 'Amigável';
  int _openSlots = 1;

  late final _dateCtrl = TextEditingController(text: _formatDate(_date));
  late final _timeCtrl = TextEditingController(text: _formatTime(_time));

  @override
  void dispose() {
    _dateCtrl.dispose();
    _timeCtrl.dispose();
    super.dispose();
  }

  String _formatDate(DateTime d) => '${_weekdays[d.weekday - 1]}, ${d.day} de
${_months[d.month - 1]}';

  String _formatTime(TimeOfDay t) =>
      '${t.hour.toString().padLeft(2, '0')}:${t.minute.toString().padLeft(2,
'0')}';

  Future<void> _pickDate() async {
    final picked = await showDatePicker(
      context: context,
      initialDate: _date,
      firstDate: DateTime(2020),
      lastDate: DateTime(2100),
    );
    if (picked == null) return;
    setState(() {
      _date = picked;
      _dateCtrl.text = _formatDate(picked);
    });
```

```
  }

  Future<void> _pickTime() async {
    final picked = await showTimePicker(context: context, initialTime:
_time);
    if (picked == null) return;
    setState(() {
      _time = picked;
      _timeCtrl.text = _formatTime(picked);
    });
  }

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.darkStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          padding: EdgeInsets.fromLTRB(20, topInset(context, 48), 20, 28 +
bottomInset(context)),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Row(
                children: [
                  CircleIconButton(
                    semanticLabel: 'Voltar',
                    onPressed: () => Navigator.of(context).pop(),
                    child: const SvgIcon(AppIcon.back),
                  ),
                  const SizedBox(width: 12),
                  const ScreenTitle('Criar jogo'),
                ],
              ),
              const SizedBox(height: 16),
              AppDropdown<String>(
                label: 'Desporto',
                value: _sport,
                items: {for (final s in sports) s.name: s.name},
                onChanged: (v) => setState(() => _sport = v ?? _sport),
              ),
              const SizedBox(height: 16),
              Row(
                children: [
                  Expanded(
                    flex: 3,
```

```
                    child: AppTextField(
                      label: 'Data',
                      controller: _dateCtrl,
                      weight: FontWeight.w600,
                      readOnly: true,
                      onTap: _pickDate,
                    ),
                  ),
                  const SizedBox(width: 12),
                  Expanded(
                    flex: 2,
                    child: AppTextField(
                      label: 'Hora',
                      controller: _timeCtrl,
                      weight: FontWeight.w600,
                      readOnly: true,
                      onTap: _pickTime,
                    ),
                  ),
                ],
              ),
              const SizedBox(height: 16),
              const AppTextField(label: 'Local', initialValue: 'Padel Club
X', weight: FontWeight.w600),
              const SizedBox(height: 16),
              _ChoiceGroup(
                label: 'Nível',
                options: levels,
                selected: _level,
                onSelected: (v) => setState(() => _level = v),
              ),
              const SizedBox(height: 16),
              _ChoiceGroup(
                label: 'Formato',
                options: formats,
                selected: _format,
                onSelected: (v) => setState(() => _format = v),
              ),
              const SizedBox(height: 16),
              Row(
                crossAxisAlignment: CrossAxisAlignment.end,
                children: [
                  Expanded(
                    flex: 3,
                    child: LabeledField(
                      label: 'Vagas abertas',
```

```
                      child: FieldBox(
                        horizontalPadding: 4,
                        child: Row(
                          mainAxisAlignment: MainAxisAlignment.spaceBetween,
                          children: [
                            _StepperButton(
                              label: '−',
                              semanticLabel: 'Menos uma vaga',
                              onTap: _openSlots > 1 ? () => setState(() =>
_openSlots--) : null,
                            ),
                            Text('$_openSlots de $_capacity', style:
AppText.body(16, weight: FontWeight.w700)),
                            _StepperButton(
                              label: '+',
                              semanticLabel: 'Mais uma vaga',
                              onTap: _openSlots < _capacity - 1 ? () =>
setState(() => _openSlots++) : null,
                            ),
                          ],
                        ),
                      ),
                    ),
                  ),
                  const SizedBox(width: 12),
                  const Expanded(
                    flex: 2,
                    child: AppTextField(
                      label: 'Preço por pessoa',
                      initialValue: '8 €',
                      weight: FontWeight.w600,
                    ),
                  ),
                ],
              ),
              const SizedBox(height: 16),
              const Spacer(),
              AppButton(
                label: 'Criar jogo',
                onPressed: () => Navigator.of(context).pushReplacement(
                  MaterialPageRoute(builder: (_) => const
GameDetailScreen()),
                ),
              ),
            ],
          ),
```

```
        ),
      ),
    );
  }
}

class _ChoiceGroup extends StatelessWidget {
  const _ChoiceGroup({
    required this.label,
    required this.options,
    required this.selected,
    required this.onSelected,
  });

  final String label;
  final List<String> options;
  final String selected;
  final ValueChanged<String> onSelected;

  @override
  Widget build(BuildContext context) {
    return LabeledField(
      label: label,
      child: Row(
        children: [
          for (var i = 0; i < options.length; i++) ...[
            if (i > 0) const SizedBox(width: 8),
            Expanded(
              child: ChoiceTile(
                label: options[i],
                selected: options[i] == selected,
                onTap: () => onSelected(options[i]),
              ),
            ),
          ],
        ],
      ),
    );
  }
}

class _StepperButton extends StatelessWidget {
  const _StepperButton({required this.label, required this.semanticLabel,
this.onTap});

  final String label;
```

```
  final String semanticLabel;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      label: semanticLabel,
      excludeSemantics: true,
      child: Material(
        color: AppColors.chip,
        borderRadius: BorderRadius.circular(10),
        clipBehavior: Clip.antiAlias,
        child: InkWell(
          onTap: onTap,
          child: SizedBox(
            width: 44,
            height: 44,
            child: Center(
              child: Text(
                label,
                style: AppText.body(
                  20,
                  weight: FontWeight.w700,
                  color: onTap == null ? AppColors.muted : AppColors.ink,
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

Ecrã 08 — Mapa

Mostra no mapa onde estão o utilizador ("Tu"), os jogadores perto e os jogos com vagas. Por cima ficam os filtros e, em baixo, um cartão com o jogo em destaque. O mapa é o desenho SVG do mockup e não um mapa real; ver a nota no fim desta secção.

Como está construído: 3 camadas num `Stack`

1. **Mapa (fundo):** um `SizedBox` de 390 × 768 (o tamanho exato do mockup) com o SVG e os marcadores nas mesmas coordenadas do HTML (`left: 167, top: 322` → `Positioned(left: 167, top: 322)`). Um `FittedBox(fit: BoxFit.cover)` estica este quadro para encher o ecrã do telemóvel, e os marcadores acompanham o mapa.
2. **Filtros (topo):** `Positioned(left: 16, right: 16, top: ...)` , fora do `FittedBox` , para manterem o tamanho real.
3. **Cartão do jogo (fundo do ecrã):** `Positioned(left: 16, right: 16, bottom: 16)` .

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Fundo	#E2E9DD, rio #CBDDE6, parques #CFE3C4, ruas brancas	`_mapSvg` (copiado do mockup, com o retângulo de fundo acrescentado)
Raio de 5 km	círculo tracejado verde com 8% de opacidade	Dentro do `_mapSvg`
"Tu"	círculo verde de 56 px, borda branca de 4 px	`_YouMarker`
Jogadores	avatar de 40 px, borda 3 px + etiqueta branca 12 px 700	`_PlayerMarker`
Clube	quadrado lima de 48 px, raio 14, borda escura 3 px + etiqueta escura	`_VenueMarker`
Cartão do jogo	branco, raio 20, padding 16	`SurfaceCard(radius: 20)`
"3 / 4"	etiqueta lima	`Tag('3 / 4', background: AppColors.lime)`
"Ver jogo"	48 px, raio 12, 16 px	`AppButton(height: 48, radius: 12, fontSize: 16)`

Comportamento

- **Ver jogo** e o toque no marcador do clube abrem o Detalhe do jogo.
- O filtro "Raio 5 km" liga e desliga; "Padel" está sempre selecionado.
- Por ser um separador do `HomeShell` , o ecrã não tem `Scaffold` : devolve um `ColoredBox` com o `Stack` .

Nota: para um mapa real com GPS, substituam o `Positioned.fill(...)` do mapa por um `GoogleMap` (pacote `google_maps_flutter`) ou um `FlutterMap` (pacote `flutter_map`, com OpenStreetMap e sem chave de API). Os filtros e o cartão ficam iguais.

`lib/screens/map_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import 'game_detail_screen.dart';

/// Ilustração do mapa (igual ao mockup). Substituir por um mapa real
/// (ex.: google_maps_flutter / flutter_map) quando houver localização.
const _mapSvg = '''
<svg width="390" height="768" viewBox="0 0 390 768" fill="none"
xmlns="http://www.w3.org/2000/svg">
<rect width="390" height="768" fill="#E2E9DD"/>
<path d="M-20 520C80 470 150 560 250 520S380 430 420 450V620C340 610 300 690
200 680S60 640-20 690z" fill="#CBDDE6"/>
<rect x="232" y="118" width="120" height="96" rx="14" fill="#CFE3C4"/>
<rect x="28" y="230" width="92" height="70" rx="14" fill="#CFE3C4"/>
<path d="M-10 190L400 150M-10 400L400 360M70-10L130 780M260-10L300 780M-10
90L400 300M150 780L400 520" stroke="#FFFFFF" stroke-width="14" stroke-
linecap="round"/>
<path d="M-10 290L400 250M180-10L200 780" stroke="#FFFFFF" stroke-width="7"
stroke-linecap="round"/>
<circle cx="195" cy="350" r="150" fill="#1F7A4D" fill-opacity="0.08"
stroke="#1F7A4D" stroke-width="2" stroke-dasharray="6 8"/>
</svg>
''';

/// 08 Mapa
class MapScreen extends StatefulWidget {
  const MapScreen({super.key});

  @override
  State<MapScreen> createState() => _MapScreenState();
}

class _MapScreenState extends State<MapScreen> {
  bool _within5km = false;
```

```
  void _openGame() => Navigator.of(context).push(
        MaterialPageRoute(builder: (_) => const GameDetailScreen()),
      );

  @override
  Widget build(BuildContext context) {
    return ColoredBox(
      color: AppColors.mapLand,
      child: Stack(
        children: [
          // Mapa desenhado em 390x768 (área do mockup) e escalado para
cobrir o ecrã.
          Positioned.fill(
            child: ClipRect(
              child: FittedBox(
                fit: BoxFit.cover,
                alignment: Alignment.topCenter,
                child: SizedBox(
                  width: 390,
                  height: 768,
                  child: Stack(
                    children: [
                      SvgPicture.string(_mapSvg, width: 390, height: 768),
                      const Positioned(left: 236, top: 228, child:
_PlayerMarker('S', 'Sebastião · 1,2 km')),
                      const Positioned(left: 40, top: 150, child:
_PlayerMarker('H', 'Helio · 3,1 km')),
                      Positioned(
                        left: 70,
                        top: 404,
                        child: GestureDetector(onTap: _openGame, child: const
_VenueMarker('Padel Club X · 1 vaga')),
                      ),
                      const Positioned(left: 167, top: 322, child:
_YouMarker()),
                    ],
                  ),
                ),
              ),
            ),
          ),
          Positioned(
            left: 16,
            right: 16,
            top: topInset(context, 52),
```

```
            child: SingleChildScrollView(
              scrollDirection: Axis.horizontal,
              child: Row(
                children: [
                  const PillChip(label: 'Padel', selected: true),
                  const SizedBox(width: 8),
                  PillChip(
                    label: 'Raio 5 km',
                    selected: _within5km,
                    onTap: () => setState(() => _within5km = !_within5km),
                  ),
                  const SizedBox(width: 8),
                  const PillChip(label: 'Jogos e jogadores'),
                ],
              ),
            ),
          ),
          Positioned(
            left: 16,
            right: 16,
            bottom: 16,
            child: SurfaceCard(
              radius: 20,
              padding: const EdgeInsets.all(16),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                mainAxisSize: MainAxisSize.min,
                children: [
                  Row(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Expanded(
                        child: Column(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            Text('Jogo de Padel', style:
AppText.display(20)),
                            const SizedBox(height: 3),
                            Text(
                              'Sáb, 3 Out · 10:00 · Padel Club X',
                              style: AppText.body(14, color:
AppColors.muted),
                            ),
                            const SizedBox(height: 3),
                            Text(
                              'Intermédio · a 2,4 km de ti',
```

```
                                style: AppText.body(14, color:
AppColors.muted),
                              ),
                            ],
                          ),
                        ),
                        const SizedBox(width: 12),
                        const Tag('3 / 4', background: AppColors.lime),
                      ],
                    ),
                    const SizedBox(height: 12),
                    AppButton(label: 'Ver jogo', height: 48, radius: 12,
fontSize: 16, onPressed: _openGame),
                  ],
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

class _YouMarker extends StatelessWidget {
  const _YouMarker();

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 56,
      height: 56,
      alignment: Alignment.center,
      decoration: BoxDecoration(
        color: AppColors.primary,
        shape: BoxShape.circle,
        border: Border.all(color: AppColors.white, width: 4),
      ),
      child: Text('Tu', style: AppText.body(15, weight: FontWeight.w700,
color: AppColors.white)),
    );
  }
}

class _MarkerLabel extends StatelessWidget {
  const _MarkerLabel(this.text, {this.dark = false});
```

```
  final String text;
  final bool dark;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 3),
      decoration: BoxDecoration(
        color: dark ? AppColors.ink : AppColors.white,
        borderRadius: BorderRadius.circular(10),
      ),
      child: Text(
        text,
        style: AppText.body(12, weight: FontWeight.w700, color: dark ?
AppColors.white : AppColors.ink),
      ),
    );
  }
}

class _PlayerMarker extends StatelessWidget {
  const _PlayerMarker(this.initial, this.label);

  final String initial;
  final String label;

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        Avatar(initial, size: 40, fontSize: 16, borderWidth: 3),
        const SizedBox(height: 4),
        _MarkerLabel(label),
      ],
    );
  }
}

class _VenueMarker extends StatelessWidget {
  const _VenueMarker(this.label);

  final String label;

  @override
  Widget build(BuildContext context) {
```

```
    return Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        Container(
          width: 48,
          height: 48,
          alignment: Alignment.center,
          decoration: BoxDecoration(
            color: AppColors.lime,
            borderRadius: BorderRadius.circular(14),
            border: Border.all(color: AppColors.ink, width: 3),
          ),
          child: const SvgIcon(AppIcon.pin, size: 24),
        ),
        const SizedBox(height: 4),
        _MarkerLabel(label, dark: true),
      ],
    );
  }
}
```

Ecrã 09 — Detalhe do jogo

Tem um cabeçalho verde-floresta com o desporto, a data e o local. Por baixo vêm 3 cartões de resumo, a lista de quem vai (com as vagas livres a tracejado) e o botão "Entrar no jogo".

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Cabeçalho	#123D29, padding 48 / 20 / 24, espaço 14	`Container(color: AppColors.forest, ...)`
Voltar	círculo #2C5A43 sem borda, seta branca	`CircleIconButton(background: forestLight, bordered: false)`
Etiqueta	"PADEL · AMIGÁVEL" 13 px 700, letter-spacing 1.2, lima	`SectionLabel(color: lime, letterSpacing: 1.2)`
Título	34 px 800, line-height 1.08, letter-spacing -0.8	`AppText.display(34, weight: w800, height: 1.08)`
Resumo	grelha de 3 colunas, espaço 8, raio 14, padding 12	`IntrinsicHeight` + `Row` + `_StatCell`

Elemento	Medidas do mockup	No código
Quem vai	linhas com padding 12 / 14, avatar de 40 px	`_ParticipantRow`
Organizador	etiqueta 13 px 700, fundo #E6ECE3	`Tag(fontSize: 13)`
Vaga livre	círculo tracejado verde de 40 px com +	`_OpenSlotRow` + `_DashedCirclePainter`
Rodapé	nota 14 px centrada + botão	`Spacer()` + `Text` + `AppButton`

Comportamento e técnicas

- **Contas automáticas:** "3 de 4" e "1 vaga livre" são calculados a partir da lista `gameParticipants` e da capacidade (`_capacity = 4`). Se acrescentarem um participante aos dados, as vagas atualizam-se sozinhas.
- **`IntrinsicHeight`** põe os 3 cartões de resumo com a altura do mais alto. "8 € / pessoa" ocupa 2 linhas e, sem isto, os outros 2 cartões ficariam mais baixos.
- **Círculo tracejado:** o Flutter não tem bordas tracejadas (`border: dashed` do CSS), por isso o `_DashedCirclePainter` desenha 14 arcos com um `CustomPainter` . Cada arco ocupa 55% do seu setor, e o resto fica vazio.
- **Entrar no jogo** abre o Check-in com `push` .

`lib/screens/game_detail_screen.dart`

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import '../data/mock_data.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import 'checkin_screen.dart';

/// 09 Detalhe do jogo
class GameDetailScreen extends StatelessWidget {
  const GameDetailScreen({super.key});

  static const _capacity = 4;
```

```
  @override
  Widget build(BuildContext context) {
    final openSlots = _capacity - gameParticipants.length;

    return AnnotatedRegion(
      value: AppTheme.lightStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Container(
                color: AppColors.forest,
                padding: EdgeInsets.fromLTRB(20, topInset(context, 48), 20,
24),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    CircleIconButton(
                      semanticLabel: 'Voltar',
                      background: AppColors.forestLight,
                      bordered: false,
                      onPressed: () => Navigator.of(context).pop(),
                      child: const SvgIcon(AppIcon.back, color:
AppColors.white),
                    ),
                    const SizedBox(height: 14),
                    const SectionLabel('Padel · Amigável', color:
AppColors.lime, letterSpacing: 1.2),
                    const SizedBox(height: 14),
                    Text(
                      'Sábado, 3 de outubro às 10:00',
                      style: AppText.display(
                        34,
                        weight: FontWeight.w800,
                        color: AppColors.white,
                        letterSpacing: -0.8,
                        height: 1.08,
                      ),
                    ),
                    const SizedBox(height: 14),
                    Row(
                      children: [
                        const SvgIcon(AppIcon.pin, size: 18, color:
AppColors.white),
                        const SizedBox(width: 6),
```

```
                        Text(
                          'Padel Club X · a 2,4 km de ti',
                          style: AppText.body(15, color: AppColors.white),
                        ),
                      ],
                    ),
                  ],
                ),
              ),
              Expanded(
                child: Padding(
                  padding: EdgeInsets.fromLTRB(20, 20, 20, 28 +
bottomInset(context)),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.stretch,
                    children: [
                      IntrinsicHeight(
                        child: Row(
                          crossAxisAlignment: CrossAxisAlignment.stretch,
                          children: [
                            const Expanded(child: _StatCell('Nível',
'Intermédio')),
                            const SizedBox(width: 8),
                            Expanded(
                              child: _StatCell('Jogadores',
'${gameParticipants.length} de $_capacity'),
                            ),
                            const SizedBox(width: 8),
                            const Expanded(child: _StatCell('Preço', '8 € /
pessoa')),
                          ],
                        ),
                      ),
                      const SizedBox(height: 18),
                      Section(
                        title: 'Quem vai',
                        child: ListCard(
                          children: [
                            for (final p in gameParticipants)
_ParticipantRow(p),
                            if (openSlots > 0) _OpenSlotRow(openSlots),
                          ],
                        ),
                      ),
                      const SizedBox(height: 18),
                      const Spacer(),
```

```
                      Text(
                        'O organizador é avisado assim que entrares.',
                        textAlign: TextAlign.center,
                        style: AppText.body(14, color: AppColors.muted),
                      ),
                      const SizedBox(height: 10),
                      AppButton(
                        label: 'Entrar no jogo',
                        onPressed: () => Navigator.of(context).push(
                          MaterialPageRoute(builder: (_) => const
CheckInScreen()),
                        ),
                      ),
                    ],
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class _StatCell extends StatelessWidget {
  const _StatCell(this.label, this.value);

  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    return SurfaceCard(
      radius: 14,
      padding: const EdgeInsets.all(12),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(label, style: AppText.body(12, color: AppColors.muted)),
          const SizedBox(height: 4),
          Text(value, style: AppText.body(15, weight: FontWeight.w700)),
        ],
      ),
    );
  }
```

```
}

class _ParticipantRow extends StatelessWidget {
  const _ParticipantRow(this.participant);

  final GameParticipant participant;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
      child: Row(
        children: [
          Avatar(participant.name.substring(0, 1), size: 40, fontSize: 17),
          const SizedBox(width: 12),
          Expanded(child: Text(participant.name, style: AppText.body(16,
weight: FontWeight.w700))),
          if (participant.organizer)
            Tag(participant.detail, fontSize: 13, padding: const
EdgeInsets.symmetric(horizontal: 10, vertical: 5))
          else
            Text(participant.detail, style: AppText.body(14, color:
AppColors.muted)),
        ],
      ),
    );
  }
}

class _OpenSlotRow extends StatelessWidget {
  const _OpenSlotRow(this.count);

  final int count;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
      child: Row(
        children: [
          const CustomPaint(
            painter: _DashedCirclePainter(color: AppColors.primary),
            child: SizedBox(
              width: 40,
              height: 40,
              child: Center(
```

```
                child: SvgIcon(AppIcon.plus, size: 20, color:
AppColors.primary, strokeWidth: 2.2),
              ),
            ),
          ),
          const SizedBox(width: 12),
          Text(
            count == 1 ? '1 vaga livre' : '$count vagas livres',
            style: AppText.body(16, weight: FontWeight.w700, color:
AppColors.primary),
          ),
        ],
      ),
    );
  }
}

/// Círculo tracejado (border: 2px dashed) da vaga livre.
class _DashedCirclePainter extends CustomPainter {
  const _DashedCirclePainter({required this.color});

  final Color color;

  @override
  void paint(Canvas canvas, Size size) {
    const strokeWidth = 2.0;
    const dashes = 14;
    final paint = Paint()
      ..color = color
      ..style = PaintingStyle.stroke
      ..strokeWidth = strokeWidth;
    final rect = Rect.fromLTWH(
      strokeWidth / 2,
      strokeWidth / 2,
      size.width - strokeWidth,
      size.height - strokeWidth,
    );
    const sweep = 2 * math.pi / dashes;
    for (var i = 0; i < dashes; i++) {
      canvas.drawArc(rect, i * sweep, sweep * 0.55, false, paint);
    }
  }
```

```
  @override
  bool shouldRepaint(_DashedCirclePainter oldDelegate) => oldDelegate.color
!= color;
}
```

Ecrã 10 — Check-in por QR Code

É o único ecrã com fundo quase preto (#0E1A14). Mostra o QR Code que o jogador apresenta no local para confirmar a presença, dentro de uma mira lima.

Correspondência com o mockup

Elemento	Medidas do mockup	No código
Ecrã	fundo #0E1A14, padding 48 / 20 / 28	`Scaffold(backgroundColor: AppColors.night)`
Fechar	círculo #22362C de 44 px, ícone X	`CircleIconButton(background: nightLight)` + `AppIcon.close`
Título	22 px 700, centrado com `padding-right: 56`	`Expanded` + `Padding(right: 56)` + `TextAlign.center`
Informação do jogo	margem 40; etiqueta lima, 24 px 700, 15 px #C9D3CB	`SectionLabel` + `Text` + `Text`
Mira	300 × 300; 4 cantos de 44 px, traço 4 px, raio 18	`_ScanCornersPainter`
Caixa do QR	branca, 256 × 256, raio 20	`Container(width: 256, height: 256)`
QR	220 × 220, cor #0E1A14	`QrImageView(size: 220)`
Texto	17 px, line-height 1.45, max-width 280, margem 32	`ConstrainedBox(maxWidth: 280)`
Botão	contorno branco de 1,5 px, colado ao fundo	`AppButton(outlined: true, foreground: AppColors.white)`

Comportamento e técnicas

- **QR Code:** o `QrImageView` (pacote `qr_flutter`) gera o QR a partir do texto `checkInCode`. Por omissão é `whoplay://checkin/padel-club-x/2026-10-03T10:00`, mas o construtor aceita outro código. Numa versão com backend, cada jogador recebe um código único para cada jogo. Os quadrados são gerados a partir do código, por isso não são iguais aos do mockup.

- **Cantos da mira:** o Flutter não junta bordas de um só lado com cantos arredondados, por isso o `_ScanCornersPainter` desenha 1 canto (linha, arco de 90°, linha) e repete-o 4 vezes espelhado com `canvas.scale(±1, ±1)`.
- **Guardar QR Code** mostra uma mensagem. Para guardar de facto na galeria, usem os pacotes `screenshot` e `image_gallery_saver` (ou parecidos).

`lib/screens/checkin_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:qr_flutter/qr_flutter.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';

/// 10 Check-in por QR Code
class CheckInScreen extends StatelessWidget {
  const CheckInScreen({super.key, this.checkInCode =
'whoplay://checkin/padel-club-x/2026-10-03T10:00'});

  final String checkInCode;

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.lightStatusIcons,
      child: Scaffold(
        backgroundColor: AppColors.night,
        body: FillScrollView(
          padding: EdgeInsets.fromLTRB(20, topInset(context, 48), 20, 28 +
bottomInset(context)),
          child: Column(
            children: [
              Row(
                children: [
                  CircleIconButton(
                    semanticLabel: 'Fechar',
                    background: AppColors.nightLight,
                    bordered: false,
                    onPressed: () => Navigator.of(context).pop(),
                    child: const SvgIcon(AppIcon.close, color:
AppColors.white),
                  ),
```

```
                const SizedBox(width: 12),
                Expanded(
                  child: Padding(
                    padding: const EdgeInsets.only(right: 56),
                    child: Text(
                      'Check-in no jogo',
                      textAlign: TextAlign.center,
                      style: AppText.display(22, color: AppColors.white,
letterSpacing: -0.3),
                    ),
                  ),
                ),
              ],
            ),
            const SizedBox(height: 40),
            const SectionLabel('Padel · Amigável', color: AppColors.lime,
letterSpacing: 1.2),
            const SizedBox(height: 6),
            Text(
              'Sáb, 3 de outubro · 10:00',
              textAlign: TextAlign.center,
              style: AppText.display(24, color: AppColors.white),
            ),
            const SizedBox(height: 6),
            Text('Padel Club X', style: AppText.body(15, color:
AppColors.softText)),
            const SizedBox(height: 36),
            SizedBox(
              width: 300,
              height: 300,
              child: CustomPaint(
                painter: const _ScanCornersPainter(color: AppColors.lime),
                child: Center(
                  child: Container(
                    width: 256,
                    height: 256,
                    alignment: Alignment.center,
                    decoration: BoxDecoration(
                      color: AppColors.white,
                      borderRadius: BorderRadius.circular(20),
                    ),
                    child: QrImageView(
                      data: checkInCode,
                      size: 220,
                      padding: EdgeInsets.zero,
                      backgroundColor: AppColors.white,
```

```
                        semanticsLabel: 'QR Code de check-in',
                        eyeStyle: const QrEyeStyle(eyeShape:
QrEyeShape.square, color: AppColors.night),
                        dataModuleStyle: const QrDataModuleStyle(
                          dataModuleShape: QrDataModuleShape.square,
                          color: AppColors.night,
                        ),
                      ),
                    ),
                  ),
                ),
              ),
              const SizedBox(height: 32),
              ConstrainedBox(
                constraints: const BoxConstraints(maxWidth: 280),
                child: Text(
                  'Mostra este QR Code no local para confirmar a tua
presença.',
                  textAlign: TextAlign.center,
                  style: AppText.body(17, color: AppColors.white, height:
1.45),
                ),
              ),
              const SizedBox(height: 32),
              const Spacer(),
              AppButton(
                label: 'Guardar QR Code',
                outlined: true,
                foreground: AppColors.white,
                onPressed: () => showInfo(context, 'QR Code guardado.'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

/// Os quatro cantos de "mira" à volta do QR Code (44px, 4px, raio 18).
class _ScanCornersPainter extends CustomPainter {
  const _ScanCornersPainter({required this.color});

  final Color color;

  @override
```

```
  void paint(Canvas canvas, Size size) {
    const length = 44.0;
    const stroke = 4.0;
    const radius = 18.0;
    const inset = stroke / 2;
    final paint = Paint()
      ..color = color
      ..style = PaintingStyle.stroke
      ..strokeWidth = stroke;

    // Canto superior esquerdo; os outros são reflexões deste.
    final corner = Path()
      ..moveTo(inset, length)
      ..lineTo(inset, radius)
      ..arcToPoint(const Offset(radius, inset), radius: const
Radius.circular(radius - inset))
      ..lineTo(length, inset);

    for (final (sx, sy) in [(1.0, 1.0), (-1.0, 1.0), (1.0, -1.0), (-1.0,
-1.0)]) {
      canvas.save();
      canvas.translate(sx < 0 ? size.width : 0, sy < 0 ? size.height : 0);
      canvas.scale(sx, sy);
      canvas.drawPath(corner, paint);
      canvas.restore();
    }
  }

  @override
  bool shouldRepaint(_ScanCornersPainter oldDelegate) => oldDelegate.color !=
color;
}
```

Executar, testar e resolver problemas

Com os 22 ficheiros criados, são 3 comandos: analisar, testar e correr.

Teste: `test/widget_test.dart`

Um teste simples que confirma que a soma dos critérios de compatibilidade (30 + 25 + 16 + 12 + 8) dá os 91% do Tchali. Não abre ecrãs, porque o `google_fonts` tentaria descarregar fontes da internet durante o teste.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:whoplay/data/mock_data.dart';

void main() {
  test('Compatibilidade do mockup soma 91%', () {
    final total = compatibilityBreakdown.fold<int>(0, (sum, e) => sum +
e.$2);
    expect(total, nearbyPlayers.first.compatibility);
  });
}
```

Verificar e correr

```
flutter analyze
flutter test
flutter run -d chrome
```

O resultado esperado é *No issues found!*, depois *All tests passed!*, e a app a abrir no Chrome. Para Android ou iOS, corram `flutter devices` e depois `flutter run -d <id do dispositivo>`.

Durante o `flutter run`, as teclas mais úteis são `r` (hot reload: aplica as mudanças do código sem perder o estado), `R` (hot restart: recomeça a app) e `q` (sair).

Lista de verificação do percurso

- [ ] Login → "Entrar" abre os Jogadores
- [ ] Login → "Criar conta" → escolher desportos → "Continuar" abre o Perfil
- [ ] Jogadores → "Ver perfil" → "Enviar convite" abre os Convites
- [ ] Convites → "Aceitar" troca os botões pelo texto verde
- [ ] Botão + → escolher data e hora → "Criar jogo" abre o Detalhe do jogo
- [ ] Mapa → "Ver jogo" → "Entrar no jogo" mostra o QR Code
- [ ] A barra de baixo troca de separador sem perder os filtros escolhidos

Problemas comuns

Sintoma	Causa	Solução
`flutter: command not found`	O Flutter não está no PATH	Usem o caminho completo (`/c/src/flutter/bin/flutter run`) ou juntem a pasta `bin` do Flutter ao PATH (Pré-requisitos)

Sintoma	Causa	Solução
`Waiting for another flutter command to release the startup lock...`	Já há outro comando `flutter` a correr (noutro terminal ou no VS Code)	Fechem o outro com `q` ou `Ctrl+C`; em último caso, terminem o processo `dart.exe` antigo
`Failed to establish connection with the web debug service`	O Chrome demorou a responder na primeira compilação, ou havia 2 `flutter run` em simultâneo	`flutter clean` e depois `flutter run -d chrome`; se continuar, `flutter run -d web-server` e abram o endereço que aparece
`PathNotFoundException ... .temp`	Ficheiros temporários apagados por outro `flutter run`	`flutter clean` e correr de novo, com um só `flutter run` aberto
As fontes aparecem diferentes (Roboto)	Sem internet na primeira execução	Liguem-se à internet; as fontes ficam em cache depois da primeira vez
`Target of URI doesn't exist` no `flutter analyze`	Falta criar algum ficheiro ou há um erro no nome	Comparem com a estrutura de pastas da Visão geral
`Couldn't resolve the package 'qr_flutter'` (ou outro)	Falta o `flutter pub get`	Corram `flutter pub get`
`RenderFlex overflowed` (faixas amarelas e pretas)	O conteúdo não cabe no ecrã	Confirmem que o ecrã usa `FillScrollView` ou `SingleChildScrollView`, como neste guia
O VS Code e o terminal usam versões diferentes do Flutter	Há 2 SDKs instalados	Fiquem só com um e apontem o VS Code para ele (*Flutter: Change SDK*)

Próximos passos sugeridos

Ligar a app a um backend (por exemplo, Firebase Auth e Firestore) através do `mock_data.dart`, fazer os passos 2 e 3 do onboarding, trocar o mapa ilustrado por um mapa real e criar um ícone da app a partir do logo (pacote `flutter_launcher_icons`).